

Constraint-Aware Interaction Generator for Synthetic E-commerce data

An End-to-End pipeline for training and evaluating interaction generators.

BEAU MIN, Amsterdam University of Applied Sciences, The Netherlands

Simulation-based studies of recommender systems are only as credible as the synthetic interaction logs that drive them. Existing log generators are dominated by process-based, agent-based, statistical, and adversarial approaches, while transformer sequence models remain largely confined to next-item prediction rather than full-session generation. This thesis investigates how, and to what extent, a constraint-aware autoregressive transformer can generate synthetic e-commerce interaction logs with high fidelity, behavioral validity, and controlled popularity bias. The generator is a multi-head transformer that jointly emits typed interaction tuples (action type, item, inter-event time), embedded in a process-based simulation wrapper and trained on the Synerise RecSys 2025 retail dataset. Three iterations of the item head (flat softmax, category-hierarchical, Singular Value Decomposition-Product Quantization (SVD-PQ) token-factored) impose successively stronger structural priors against popularity collapse, and a hard-constraint validity layer reports pre- and post-repair violation rates, keeping generation assumptions auditable. A preregistered five-seed protocol assesses distributional fidelity, behavioral validity, and popularity bias against a Markov baseline and real-continuation reference. The transformer dominates the baseline on session shape (Kolmogorov-Smirnov (KS) 0.10–0.19 vs. 0.95) with zero post-repair constraint violations at a bounded purchase-exposure repair cost. The structural priors cut popularity over-concentration (Δ Gini 0.23 $\rightarrow$ 0.05) and calibrate mainstream-user popularity bias to the real level, while niche-user amplification persists at $\approx 13\times$ across all iterations. This residual bias is measured on every run and reported as a release-gated artefact rather than silently corrected: the simulator’s contract is faithfulness to the logged traffic, biases included, and the real-continuation reference isolates what the generator itself adds.

CCS Concepts: • **Information systems** $\rightarrow$ **Recommender systems**; • **Computing methodologies** $\rightarrow$ *Neural networks; Modeling and simulation.*

Additional Key Words and Phrases: recommender systems, synthetic data, simulation, transformer, interaction generation, evaluation protocol

1 Introduction

1.1 Context

Recommender systems (RS) are frequently developed and evaluated on static historical logs: practical but methodologically incomplete. When recommendation policies change, user exposure changes. When exposure changes, user behavior changes. When behavior changes, future training data changes. Static snapshots therefore fail to capture the feedback dynamics that emerge in iterative deployment, and offline evaluation on fixed logs can look convincing while missing them. This challenge is repeatedly identified in the recommender simulation literature as a fundamental reason for interest in synthetic interaction generation as a methodological tool [7, 10, 36].

1.2 Problem Statement

The RS simulation literature is active but methodologically fragmented. Existing approaches include process-based and framework-oriented simulators [4, 26, 27], adversarial and statistical generators [3, 42], and configurable data synthesis tools [18, 35]. These lines provide important building blocks but also reveal recurring issues: heterogeneous assumptions, inconsistent metric usage, and weak comparability across studies [7, 36].

The project context of this thesis is a two-component recommender system simulator developed at the Amsterdam University of Applied Sciences (AUAS) Centre for Market Insights (CMI). An arrival component is already operational,

Author’s Contact Information: Beau Min, beau.min@hva.nl, Amsterdam University of Applied Sciences, Amsterdam, The Netherlands.

while an interaction generator is the open problem. The prior implementation couples a first-order Markov chain with an Long Short-Term-Memory (LSTM) that functions as a feature extractor for item ranking, not a learned sequence generator. Interaction generation is therefore a patchwork of a recommender, a Markov chain, and hardcoded session logic; no single learned model generates coherent interaction sequences end-to-end.

1.3 Existing Work

Sequential recommendation has demonstrated strong transformer performance, with SASRec [23], BERT4Rec [37], and GRU4Rec [16] marking the progression from recurrent to attention-based methods. But prediction success does not automatically transfer to synthetic interaction generation [21]: generator design requires distinct choices around output structure, validity constraints, temporal handling, and evaluation criteria (Section 2).

1.4 Research Gap

Across the RS simulation literature reviewed, no work uses a non-Large Language Model autoregressive transformer as the primary architecture for generating enriched interaction event logs[10, 36]: the simulation branch relies on process-based, agent-based, statistical, or adversarial approaches, while the sequential-recommendation branch applies transformers exclusively to next-item prediction. This thesis is positioned at the intersection of these two lines.

1.5 Proposal and Research Questions

This thesis proposes a hybrid architecture: a multi-head autoregressive transformer embedded in a process-based simulation wrapper, generating typed event sequences with explicit hard constraints. The contribution is both *technical* (the architecture) and *methodological* (a reproducible evaluation protocol for simulator quality claims along fidelity, validity, and popularity bias).

The work is guided by the following main research question and sub-questions:

How and to what extent can a constraint-aware autoregressive transformer interaction generator produce synthetic interaction logs with high fidelity, validity, and controlled popularity bias for recommender-system training and evaluation?

- **SQ1:** How should interactions be formally defined for simulation purposes, covering event fields, temporal structure, and session dependencies?
- **SQ2:** What architectural properties does an interaction generator need to capture realistic interaction dynamics, including sequential dependence, action–item coherence, and temporal structure?
- **SQ3:** Which generator implementations and baseline policy are feasible within thesis scope for a fair and executable comparison?
- **SQ4:** Which evaluation protocol makes “better than baseline” falsifiable and reproducible?

1.6 Contributions

The main contributions of this work are:

- (1) A multi-head autoregressive transformer architecture for typed event tuple generation, with factored heads for action type, item, and inter-event time.
- (2) A reproducible, multi-axis evaluation protocol covering fidelity, validity, and popularity bias, with variance reporting across seeds.

2 Background

2.1 Theoretical Foundations

2.1.1 Interaction as Typed Temporal Event Sequences. Across the RS simulation papers reviewed for this work, “interaction” is more than a $\langle \text{user}, \text{item}, \text{rating} \rangle$ tuple, yet each work captures only part of the richer record. Accordion models $(\text{time}, \text{user}, \text{item}, \text{outcome})$ events in continuous time but attaches a single outcome mark rather than a typed action vocabulary [27]; UserSim imitates feedback on a system-proposed item, without timestamps or sessions [42]; SIREN and T-RECS model stateful user-item processes but advance in discrete rounds with experimenter-specified choice rules rather than behavior learned from logs [4, 26]; DataGenCARS formalizes context but has no session or event-order structure [35]. The convergence of evidence supports a two-level interaction definition (event-level typed tuples grouped into sessions, with user state carried across sessions via history) that none of the reviewed simulators operationalizes in full. This definition, and the gap it exposes, directly motivates the multi-head prediction architecture of Section 6.

2.1.2 Sequential Dependence as a Modeling Requirement. Realistic interaction sequences are not independent and identically distributed events: a user who views an item, adds it to cart, then removes it exhibits session-level dependence that first-order Markov chains cannot capture reliably[23]. Chaney [7] identifies missed behavioral dependencies as a central simulation validity risk. This motivates sequence-capable generators and justifies the autoregressive design of the proposed transformer.

2.1.3 Adversarial Generation: Utility and Risk Tradeoffs. Generative Adversarial Network (GAN)-based user simulators provide utility-oriented evidence [3, 42] but carry documented limitations: mode collapse and optimization instability [8], persistent evaluation ambiguity [13], and, for discrete sequential generation, gradient-flow workarounds that compound architectural complexity [19]. Autoregressive transformers with softmax heads handle discrete token generation natively, a structural advantage for this task.

2.1.4 Multi-Axis Evaluation as a Scientific Requirement. Jordon et al. [21] argue that utility-only validation is insufficient: synthetic data that enables good downstream models may still be statistically implausible. Single-metric evaluation is systematically incomplete [13], metrics across simulators are heterogeneous and non-comparable [36], and standardized evaluation tools are absent [25]. This is the basis for the multi-axis evaluation used here.

2.1.5 Reproducibility and Assumption Explicitness. T-RECS emphasizes repeated-run variance analysis as a scientific requirement [26], SimuRec identifies reproducibility as a missing community standard [10], and Chaney [7] argues simulation conclusions are credible only when modeling assumptions are explicit and tested. This thesis operationalizes these through fixed seeds, deterministic preprocessing, a hard-constraint validity layer that makes generation assumptions explicit and measurable, and five-seed variance reporting.

2.2 State of the Art

2.2.1 Recommender System Simulation Landscape. The simulation literature lacks standardized head-to-head comparisons and shared quality-control standards, so results across simulators are rarely directly comparable[10, 36].

Process-based and framework-oriented simulators form one branch: SIREN [4] simulates longitudinal recommender effects in online news with explicit feedback dynamics, T-RECS [26] lowers the barrier to long-horizon simulation through modular agent-based design and multi-run confidence intervals, and Accordion [27] models user visits as

inhomogeneous Poisson processes with marked events. These tools establish the system-level context this thesis operates within, while leaving event-level generation to domain assumptions or simplified rules.

Adversarial and statistical generation form a second branch: UserSim [42] applies a supervised GAN to simulate user behavior for offline reinforcement learning, Bobadilla et al. [3] generate parameterized collaborative filtering datasets via GAN, and DataGenCARS [35] and Jakomin et al. [18] provide configurable statistical generators for context-aware and stream-based interaction data.

2.2.2 Sequential Modeling and Transformers. Transformers have proven dominant for next-item prediction in sequential recommendation. SASRec [23] established causal self-attention for sequential RS, demonstrating that attention-based models substantially outperform Markov chains and Recurrent Neural Networks (RNN) on next-item prediction benchmarks. BERT4Rec [37] extended this with masked training objectives for bidirectional context modeling. GRU4Rec [16] is the canonical RNN predecessor that transformers have consistently outperformed on longer sequences[23].

However, next-item prediction and synthetic log generation are distinct tasks: prediction asks what item comes next given history, generation must produce a full, distributionally realistic session from scratch, with fundamentally different output structure and success measures. This thesis applies transformer sequence modeling to generation; the architectural transfer is strongly motivated but cannot be assumed automatic, and generated logs require evaluation along fidelity and validity axes that next-item accuracy does not capture.

2.3 Stakeholder Analysis

Academic supervisor (Diptish Dey, CMI). Diptish Dey is the primary owner of this project. His stake is a working, validated interaction generator that integrates with the existing arrival simulator; timely delivery and a defensible evaluation methodology matter because the simulator is part of ongoing CMI research.

CMI researcher (Debarati Bhaumik). Debarati Bhaumik, CMI researcher and co-author of the popularity-bias metric suite adopted in Section 7.4 [5], holds the stake behind the basket-level evaluation: her request for per-user recommendation baskets — a capability the pipeline initially lacked — drove the basket-generation protocol and made the group-level bias measurement applicable to this generator (Section 3).

CMI research group. The broader CMI research group has a practical stake in the technical output: a reusable simulator component and a documented evaluation protocol colleagues can apply or extend without rebuilding from scratch.

CMI researchers (operator end-user). The immediate end-user is a CMI researcher who runs the simulator: configures an experiment, executes the pipeline, inspects the metrics. Their stake is operational rather than methodological. Decisions made for this end-user: the configuration-file-driven experiment runner, the four-stage artefact-based pipeline (any single stage re-runnable without re-running its predecessors), and a single shared evaluation harness for both generators so cross-generator comparisons need no per-experiment glue code.

Recommender systems research community. The research community’s interest is in reusable evaluation protocols, open-dataset compatibility, and reproducible baselines that support cumulative scientific progress.

Dataset provider (Synerise / RecSys 2025 challenge). Synerise’s interest is in demonstrating the richness and utility of their publicly released dataset; it has no active supervisory or evaluative role in this project.

Downstream users of the recommender system. The Synerise logs represent real user populations. These anonymous users have an interest in synthetic data not amplifying distributional biases present in the real logs, such as popularity skew: the stake behind the bias metrics and the ethical requirements of Section 4. This research neither re-identifies nor profiles any individual.

Conflicting interests and prioritization. The interests above diverge on two points. First, the supervisor’s interest in timely delivery is in tension with the research community’s interest in broad generalizability: a wider baseline portfolio (GRU- or GAN-based) would strengthen the community contribution but risks the timeline. Resolved in favor of the supervisor’s constraint: scope fixed to transformer versus Markov, extensions named as future work. Second, Synerise’s interest in showcasing dataset richness is in tension with downstream users’ interest in not amplifying bias, since a generator optimized purely for distributional fidelity faithfully reproduces the training data’s popularity skew. Resolved by including explicit bias metrics rather than treating fidelity-to-data and fairness-to-users as equivalent goals.

2.4 Current Situation

The interaction generator currently in use within the CMI simulator is a patchwork of three independent mechanisms (Section 1): a recommender selects items, a Markov chain selects action transitions, and hardcoded session logic glues them together, yielding implausible session structures and insufficient coverage. The proposed transformer replaces all three with one autoregressive model whose factored heads emit action, item, and inter-event time jointly, evaluated against the existing mechanism (Section 7).

2.5 AI Considerations

2.5.1 Suitability of a Learned Model. Rule-based approaches, such as handcoded Markov chains with manually specified valid transitions, require exhaustive elicitation of every valid behavioral path and cannot generalize to unseen contexts or long-range session dependencies; learned models extract transition probabilities from data and capture dependencies fixed rules cannot express [7, 27].

2.5.2 Autoregressive vs. Non-Autoregressive Generation. Parallel (non-autoregressive) generation produces all output tokens simultaneously, offering higher throughput[14], but requires conditional-independence assumptions across output positions that are incompatible with the documented behavioral dependencies: a generated action must condition on prior actions, and a generated item on the current action type. Violating these at generation time would undermine both the constraint layer’s validity and the output log’s fidelity [7]. Autoregressive generation is therefore the appropriate choice, with throughput as an acknowledged tradeoff.

Among learned approaches, a transformer is preferred over simpler sequence models: the factored multi-head design enables joint action–item–time modeling, and self-attention captures long-range session dependencies more effectively than first-order chains or recurrent architectures [16, 23, 37].

2.5.3 Degree of Automation and Societal Considerations. Full automation of the generation pipeline is appropriate here because the output is synthetic training data rather than decisions affecting real users directly; no human-in-the-loop is required at generation time. The hard-constraint validity layer keeps the automation explicit and auditable: generation assumptions are documented as constraint rules extracted from real data, and violations are counted and reported.

Recommender systems trained on synthetic interaction data influence what products, content, and services real users are exposed to at scale. If the generator amplifies the popularity skew in its training data, the harm lands unevenly:

users with niche taste see their preferences progressively overwritten by mainstream items (the measured amplification reaches $\approx 13\times$ their historical popularity level, Section 8.3), long-tail producers lose exposure to bestsellers, and the marketplace concentrates. The most significant risk is feedback-loop amplification [7]: a biased generator produces biased training data, which trains a biased recommender, whose logs retrain the generator.

Reporting alone does not neutralize this risk, but neither would silent debiasing by default: the pipeline’s contract is to mimic production traffic, and synthetic data that quietly corrects the distribution no longer represents the system it simulates. The response is therefore layered. The recommended configuration mitigates by construction, as the structural priors of Iterations 2–3 (Section 6.3) cut popularity over-concentration ($\Delta\text{Gini } 0.23 \rightarrow 0.05$) without any fairness knob; explicit mitigation is operable per run (requirement E2); and release is gated on human inspection of the bias report (requirement E3). One boundary remains: the real logs themselves record behaviour under a deployed recommender, so fidelity to the logs is not fidelity to unobserved preference; all claims are scoped to the logged world (Section 7.5). Within that logged world, the bias the generator itself adds remains identifiable: the real-continuation reference fixes the organic drift level, and amplification is measured as the gap above it (Section 7.4).

3 Methodology: Research Design

3.1 Research Approach

The project follows a design-and-evaluate methodology across three phases: specification (interaction schema, simulator interfaces, evaluation protocol), modeling (Markov baseline and transformer generator), and evaluation (the fidelity–validity–popularity–bias protocol with statistical reporting).

The three phases were not executed as a single pass; the prototype was iterated through an asynchronous, artefact-driven stakeholder loop. Completed pipeline stages were handed over to CMI, where supervisor Diptish Dey executed them within the CMI simulation environment and integrated them with his own tooling — including adapting the generator’s basket-emission cadence to his simulator — with the pipeline’s HTML evaluation reports as the shared artefact for recurring supervision checkpoints. Two evaluation-defining changes came out of this loop. First, CMI researcher Debarati Bhaumik requested per-user recommendation baskets, a capability the pipeline initially lacked; the anchored basket-generation protocol of Section 7.4 was built in response and became the substrate of the bias axis. Second, the pre-registered downstream-utility axis Train-Synthetic-Test-Real (TSTR), already undermined by its measured failure modes (Appendix B.1), was replaced on supervision feedback by the group-level popularity–bias suite the CMI group itself published [5], so the evaluation measures what the simulator’s operators measure in their own research.

3.2 Experimental Setup

All training and evaluation runs are executed on a single workstation with one NVIDIA GeForce RTX 4070 Ti GPU (12 GB VRAM, driver 595.71.05), running CachyOS Linux (kernel 7.0.3) with CUDA 13.2. Python dependencies are pinned in `uv.lock` and resolved by the `uv` package manager; the versions relevant to the experiments are PyTorch 2.11.0, NumPy 2.4.4, Polars 1.39.3, and pandas 3.0.2.

Multi-seed reporting uses seeds $\{42, \dots, 46\}$, reused for final evaluation. Preprocessing, cleaning, sessionization, and split steps depend only on the input parquets and configured cutoffs, and produce byte-stable output artefacts.

4 Methodology: Requirements

4.1 Legal Requirements

The Synerise RecSys 2025 dataset is released publicly under the MIT license of the challenge terms[38] and consists of anonymized behavioral logs with no personally identifiable information. Processing anonymized behavioral data for academic research is permissible under the General Data Protection Regulation (GDPR) Art. 89 scientific-research exemption [12]. Generated synthetic data carries minor re-identification risk: vocabulary capping reduces the item space, and all outputs are population-level statistics rather than individual records.

4.2 Ethical Requirements

Synthetic interaction data trained on real logs inherits that data’s distributional properties: popularity skew, temporal selection effects, and potential demographic proxy signals [7, 21]. The feedback-loop risk this poses for downstream end users (Section 2.5) translates into three requirements, owned by the downstream-user stake:

- E1. Every generated batch is scored on the popularity-bias metric group as part of the standard evaluation.
- E2. Mitigation is operable by the end user: configuration-level fairness parameters trade fidelity against amplification without code changes (Section 4.7).
- E3. The bias report acts as a release gate, inspected by a human before synthetic data is handed downstream.

These controls make inherited skew transparent and controllable, not absent. The pipeline must not be used for individual-level profiling or decisions; use is restricted to aggregate training-data generation.

4.3 Organizational Requirements

The project is conducted by a single developer within the thesis timeline; the deliverable is the thesis plus a demonstrator pipeline. Recurring supervision meetings with supervisor Diptish Dey provide structured checkpoints alongside the asynchronous artefact-driven feedback loop (Section 3).

A second group concerns adoption. The pipeline must be operable by a researcher other than its developer: configuration-driven runs, no code changes, deterministic re-runs on a fixed seed (the operator stake of Section 2.3). The staged artefact design must survive handover, letting any stage be re-run independently. And requirement E3 becomes organizational procedure: a human review of the bias report before generated data is released for downstream training.

4.4 Functional Requirements

The generator must produce typed event tuples conforming to the schema in Table 6, respecting the five-action vocabulary; sessions begin on an arrival signal from the arrival simulator and terminate on an End-Of-Session (EOS) token or a length cap; and all generated sessions must pass the validity layer before output. The generator must also support anchored per-user conditional generation, producing the recommendation baskets of the bias evaluation (Section 7.4); this requirement entered through the stakeholder loop of Section 3.

4.5 Technical Requirements

The model must be trainable on a single GPU within the timeline, with vocabulary capping ensuring softmax tractability. Evaluation infrastructure must be deterministic (fixed seeds, configuration-driven runs, all hyperparameters logged) and reproducible across multiple seeds, and the generator deployable within the containerized simulation environment alongside the arrival simulator.

4.6 Stakeholder Traceability

Each requirement traces to a stakeholder wish of Section 2.3: the supervisor’s working, integrated generator maps to the functional contract and technical deployability; Debarati Bhaumik’s basket request to the conditional-generation requirement; the operator’s no-code-changes wish to the organizational operability requirements; downstream users’ interest in non-amplified bias to E1–E3; the research community’s reproducibility interest to the technical determinism requirements; and Synerise’s release terms to the legal requirements.

4.7 Validation of Requirements

The five categories of requirements above were validated as follows.

Legal. The MIT licence of the public release permits use, modification, and redistribution, with no separate data-use agreement[38]. Compliance follows from (i) operating only on the publicly released parquets, (ii) processing no PII (PII) fields (`client_id` and `sku` are opaque integers), and (iii) restricting all reported outputs to population-level aggregates.

Ethical. Making bias inheritance visible rather than silent was validated by computing the bias-metric group on every generated batch and using it as a development signal during iteration. Two hyperparameters were added in direct response to bias readings: an inverse-frequency item-Cross Entropy (CE) weight (`item_loss_freq_weight_alpha`) [9] and a Menon-style category logit adjustment (`cat_logit_adj_tau`) [29], validating requirement E2. The E3 gate is validated by the evaluation emitting the bias report as a standard artefact of every run.

Organizational. The timeline constraint was validated by delivery against the supervision team’s shared planning board, with recurring check-ins throughout. Operability by a non-developer was validated in practice: the supervisor ran and adapted the pipeline within the CMI environment (Section 3).

Functional. The functional requirements decompose into a behavioral contract and a schema contract. The behavioral contract is validated automatically at generation time by the constraint validity layer (Section 6.4), with pre- and post-repair violation rates reported in Section 7.3, making the constraint check measurable.

Technical. Validated by direct measurement: GPU memory stayed below 4 GB during training and 6 GB during evaluation on the 12 GB RTX 4070 Ti, so the single-GPU requirement is met with headroom. Determinism is validated by byte-stable preprocessing artefacts across re-runs; multi-seed reproducibility by the five-seed protocol (Section 7.1).

5 Methodology: Data

5.1 Dataset Description

The experiments use the Synerise RecSys 2025 retail e-commerce dataset: 225,224,262 timestamped interaction events collected over 168 days, from 23 June 2022 to 8 December 2022, plus a product metadata table. The interaction schema comprises five event types (`page_visit`, `search_query`, `add_to_cart`, `remove_from_cart`, `product_buy`), released as five event parquet files and one `product_properties` file.

The events are real behavioural data, recorded in production and released anonymized for the RecSys 2025 challenge, not constructed for research. Representativity is therefore bounded by the source: one retail platform, one 23 June–8 December window (including the seasonal ramp-up of Figure 1), and behaviour logged under whatever recommendation the platform had deployed. This bound matters doubly here, since a generator trained on these logs can at best be as

Table 1. Raw action distribution in the Synerise RecSys 2025 dataset. The 88.6% page-visit share means raw event likelihood is dominated by browsing; the monetizing actions the simulator exists to model contribute under 5% of the stream.

Action type	Count	Share
page_visit	199,451,980	88.56%
search_query	13,223,769	5.87%
add_to_cart	7,541,117	3.35%
remove_from_cart	2,688,894	1.19%
product_buy	2,318,502	1.03%
Total	225,224,262	100.00%

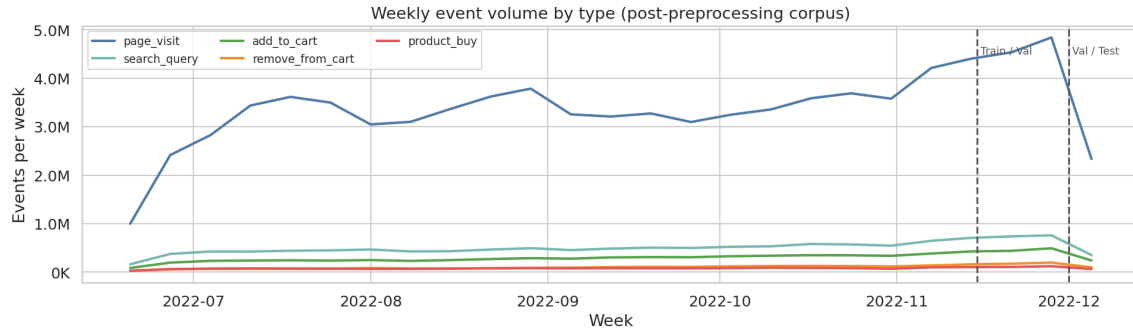


Fig. 1. Weekly event volume by type on the post-preprocessing corpus, with train/validation and validation/test split boundaries marked. Event volume is non-stationary across the observation window; the held-out splits lie in the highest-volume period.

representative as the logs themselves; what the data does and does not represent is treated as a first-class limitation (Sections 5.3 and 2.5).

The raw action distribution is imbalanced toward browsing activity (Table 1): page visits account for 88.56% of all logged events, purchases for only 1.03%, so the generator must learn realistic transition behavior in a corpus dominated by low-value, high-frequency browsing. The temporal structure is shown in Figure 1: event volume rises substantially over the observation window, and the held-out periods overlap with the densest portion of the corpus.

The product metadata table contains 1,534,050 rows with four null-free fields (sku, category, price, name), spanning 6,912 categories and 100 price buckets, with complete coverage for training Stock Keeping Units (SKUs), suitable for feature-augmented sequence modeling and for extracting item-category constraint structure.

The item distribution is long-tailed. Under the realized training split, 9,338,690 item-bearing events span 1,305,867 unique SKUs; three orders of magnitude separate the vocabulary size K required for 50% coverage from that for 99% (Table 2, Figure 2). The long-tail structure recurs per action type (Figure 3): add_to_cart, remove_from_cart, and product_buy all follow the same heavy-tailed shape over their SKU ranks, with strongly correlated frequencies across action types.

The stream is also structured at the session level. Under a priority-based labelling (purchase $\succ$ cart $\succ$ search $\succ$ browse), the training split decomposes into 2.0% pure browse, 32.8% search-only, 45.2% cart-without-checkout, and 19.9% purchase sessions (Figure 4; conversion funnel 100% $\rightarrow$ 98.0% $\rightarrow$ 65.2% $\rightarrow$ 19.9%), and 26.2% of training users complete at least one purchase.

Table 2. Vocabulary size K needed to cover a target fraction of training item-bearing events. The realized pipeline cap of $K = 200,000$ covers 71.71%.

Target coverage	K (number of SKUs)
50%	45,456
80%	338,307
90%	654,304
95%	887,771
99%	1,212,481

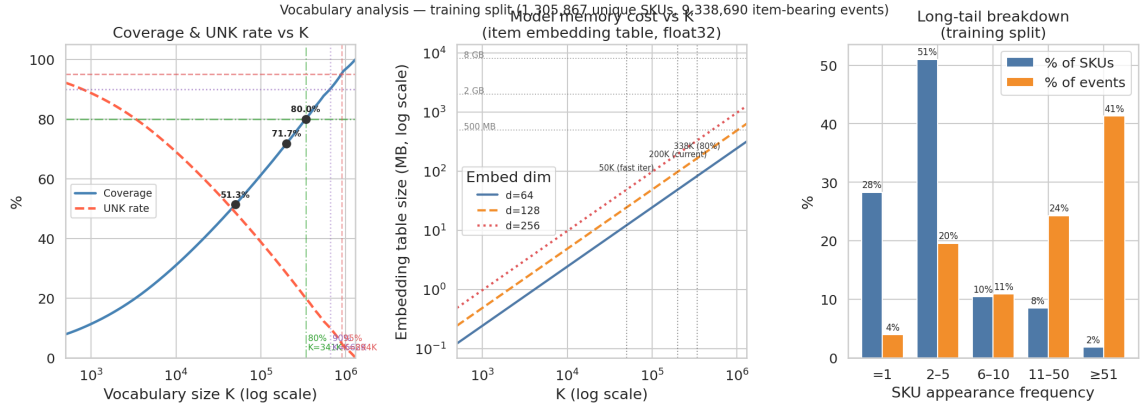


Fig. 2. Vocabulary analysis on the training split. Left: cumulative coverage and unknown-token (UNK) rate as a function of K on a log scale, with the configured pipeline cap and 80%/90%/95% reference points marked. Middle: float32 embedding-table size in MB versus K for embedding dimensions $d \in \{64, 128, 256\}$. Right: the long-tail breakdown of SKU frequency, contrasting the share of SKUs against the share of events they account for. Coverage grows logarithmically in K while memory grows linearly, so each SKU added beyond the 200K cap buys vanishing coverage at constant cost.

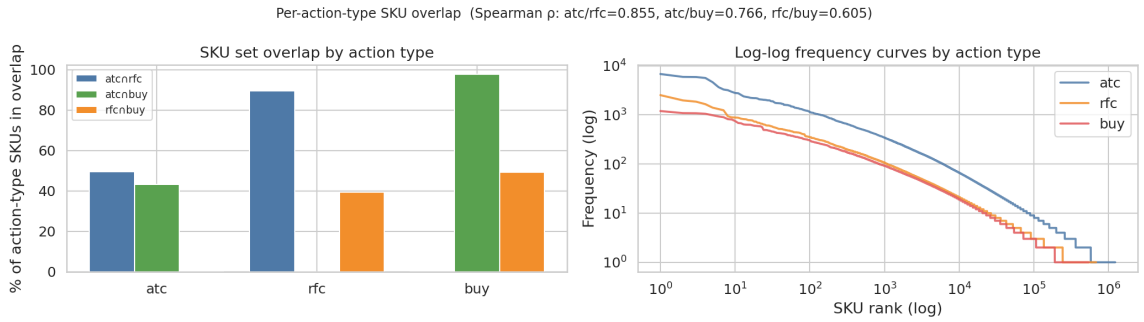


Fig. 3. Per-action-type SKU distributions on the training split. Left: pairwise SKU set overlaps. Right: log-log frequency curves for add_to_cart, remove_from_cart, and product_buy, all showing the same approximate power-law decay. The popularity skew is intrinsic to every action type rather than a browsing artefact; this is the long tail any generator trained on these logs inherits.

The dataset is well-suited for evaluating interaction generation: a temporally ordered event stream with a domain-specific behavioral funnel spanning browsing, search, cart management, and purchase. The constraints extracted from it

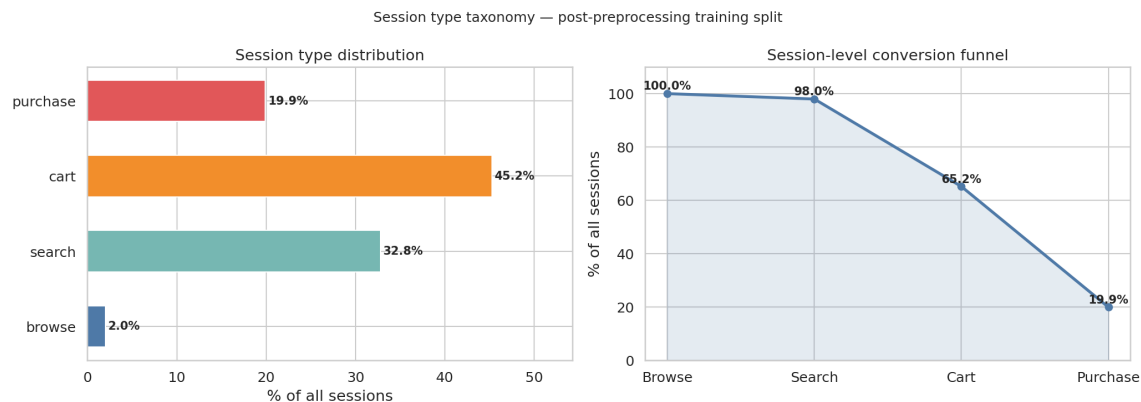


Fig. 4. Session type taxonomy and conversion funnel on the training split. Sessions are labelled by their highest-priority event. The funnel makes explicit that purchase-bearing sessions are a one-in-five minority that any session generator must reproduce in the right proportion rather than collapse into the dominant browse/search/cart modes.

are, however, retail-specific: conventions of e-commerce interaction data, not domain-agnostic rules that would transfer to media, news, or social feeds.

5.2 Preprocessing Pipeline

The preprocessing pipeline (`preprocess.py`, `ingestion/dataset.py`) runs in six stages; the full step-by-step mechanics are in the *Ontwikkeling* section of the repository wiki [30].

- (1) **Schema normalization.** The five raw event parquet files are merged into a unified interaction table keyed on `client_id`, `timestamp`, `event_type`, and `sku`, with category and price joined onto item-bearing events from `product_properties`.
- (2) **Deduplication and timestamp cleanup.** Exact duplicate item-bearing events (same `client_id`, `timestamp`, and `SKU`) are removed (696,271 events across the three item actions), while same-user same-timestamp events with different SKUs are kept as plausible multi-item basket actions. Timestamps are bounded to the dataset window (23 June–8 December 2022).
- (3) **Sessionization.** User timelines are segmented at a 30-minute inactivity threshold[15], chosen from the observed gap distribution (Figure 6): the inter-event-gap Cumulative Distribution Function (CDF) is essentially flat over 20–45 minutes, and varying the cutoff across {20, 30, 45} minutes changes the session count by under 5.5% while preserving median and tail percentiles (Table 3, Figure 7). Sessions containing only `page_visit` events are then dropped as carrying no item signal.
- (4) **Time-aware split by session start.** Train/validation/test assignment is by session start time, not event time or user: a per-event split would leak the session-prefix signal across the boundary[28], and a user or random split would not match the simulator’s forward-in-time deployment regime. Sessions starting before 15 November 2022 go to training, 15 November–1 December to validation, and on or after 1 December to test (145/16/8 days), placing the dense, behaviourally distinct late-season traffic (Black Friday, early holiday shopping) on the held-out side. The split is deterministic given the cutoffs; realized sizes are in Table 4.

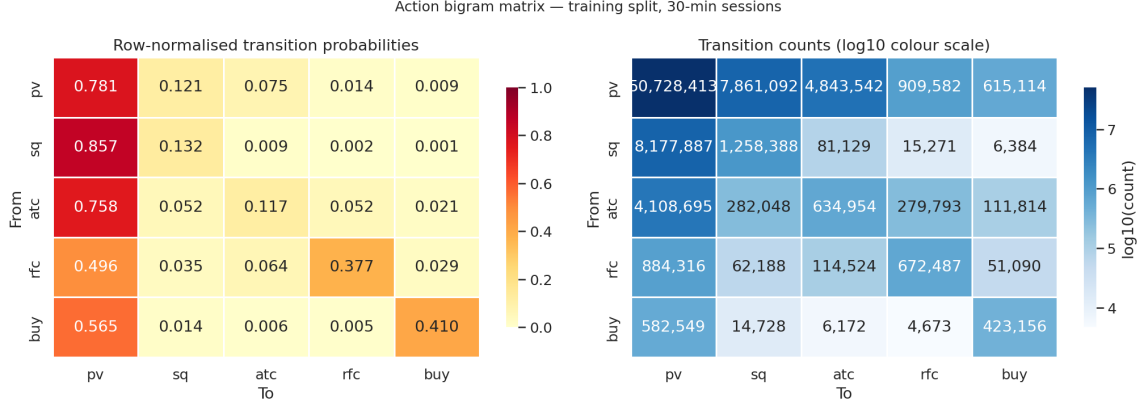


Fig. 5. Action bigram structure on the training split (30-minute sessions). Left: row-normalised transition probabilities; rows sum to one. Right: the same matrix as raw counts on a $\log_{10}$ colour scale. All 25 bigrams are observed, but six fall below the 1% row-probability threshold and are treated as soft rather than hard constraints in the generation pipeline.

Table 3. Sessionization sensitivity on the post-preprocessing corpus. The 30-minute threshold is the chosen pipeline default; baseline percentages are relative to it.

Threshold	Sessions	vs. baseline	Mean length	p90	p99
20 min	7,172,520	+5.47%	15.14	34	106
30 min	6,800,479	(baseline)	15.97	35	112
45 min	6,688,705	−1.64%	16.24	36	115

Table 4. Realized split sizes after preprocessing, page-visit-only session removal, and session-start-based splitting. Average session length is near-stable across splits (16.2 vs. 15.0), so the held-out periods differ mainly in catalogue (Table 5) rather than in session structure.

Split	Events	Sessions	Avg. session length
Train	88,170,139	5,440,150	16.21
Validation	13,689,549	910,386	15.04
Test	6,757,989	449,943	15.02
Total	108,617,677	6,800,479	15.97

- (5) **Vocabulary capping.** The item vocabulary is capped at the $K = 200,000$ most frequent training SKUs (plus a shared zero index for padding, non-item events, and Out-Of-Vocabulary (OOV items), covering 71.71% of training item-bearing events (Table 2, Figure 2). Event-level OOV rises sharply on the held-out splits (Table 5), reflecting strong temporal item drift.
- (6) **Constraint rule extraction.** Valid action transitions and exposure-style item prerequisites are extracted from the training split. All 25 action-bigram pairs occur at least once, but six fall below a 1% row-probability threshold and are treated as soft rather than hard constraints (Figure 5). Exposure prerequisites are summarised in Figure 8.

Finally, the held-out splits are meaningfully harder: Figure 9 confirms the action mix is preserved across splits while the user-overlap analysis shows most held-out users are cold-start.

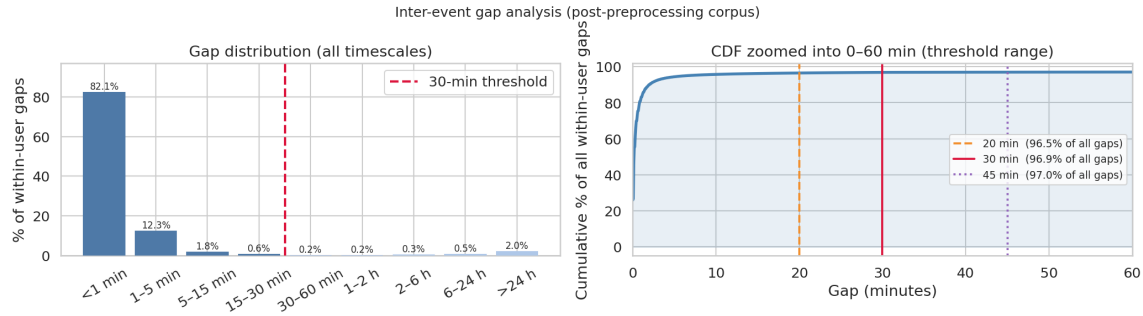


Fig. 6. Within-user inter-event gap distribution on the post-preprocessing corpus. Left: gap distribution at all timescales, with the 30-minute sessionization threshold marked. Right: CDF zoomed into 0–60 minutes; vertical lines mark the candidate thresholds {20, 30, 45} minutes used in the sensitivity analysis. The cut falls in a near-empty region, as only 0.5 percentage points of all gaps lie between 20 and 45 minutes, which is why sessionization is insensitive to the exact threshold.

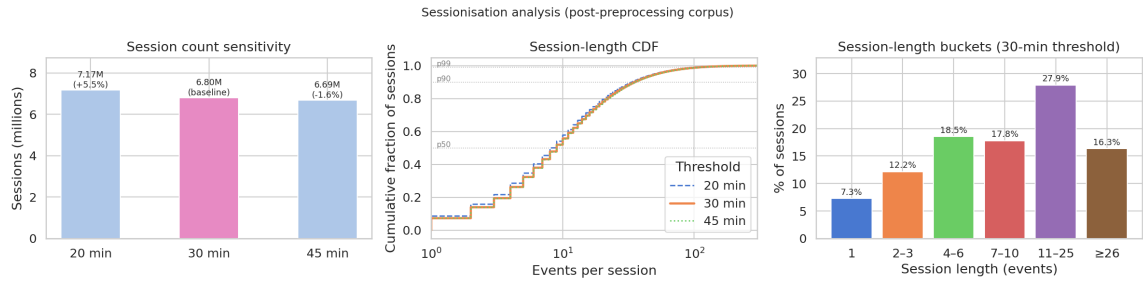


Fig. 7. Sessionization sensitivity. Left: total session count for the three candidate thresholds. Middle: session-length CDF, log-scaled on events per session. Right: bucketed session-length distribution under the chosen 30-minute threshold. The length distributions are nearly indistinguishable across thresholds, so downstream results are not artefacts of the 30-minute choice.

Table 5. Event-level OOV rate per split under the train top- K item vocabulary ($K = 200,000$). The OOV rate roughly doubles from train to test, indicating strong temporal item drift.

Split	OOV rate (item-bearing events)
Train	28.29%
Validation	42.90%
Test	50.01%

5.3 Limitations and Trade-offs

Using an existing public dataset trades reproducibility and scale for control over schema, granularity, and sampling: 225 million events over 168 days would be infeasible to collect within an MSc timeline, and a public benchmark lets future work reproduce the preprocessing against the same source artefacts. The accompanying limitations are equally direct, and several are quantitative consequences of analyses already presented above.

Vocabulary truncation is lossy. The cap $K=200,000$ covers 71.7% of training item-bearing events; the remaining 28.3% map to the shared zero index, and reaching 90%/95% coverage would require $\sim 650k/\sim 890k$ explicit SKUs (Table 2). The

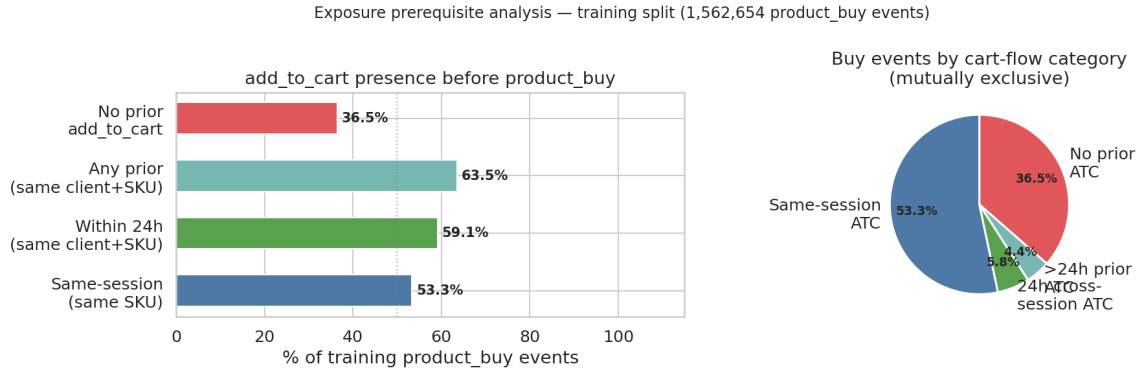


Fig. 8. Exposure prerequisite analysis on the training split. Left: share of product_buy events that have a prior add_to_cart of the same SKU, under increasingly relaxed scopes (same session, same client within 24 h, any prior). Right: the same purchases broken down into mutually exclusive cart-flow categories. Roughly one third of purchases have no prior same-SKU cart event at all.

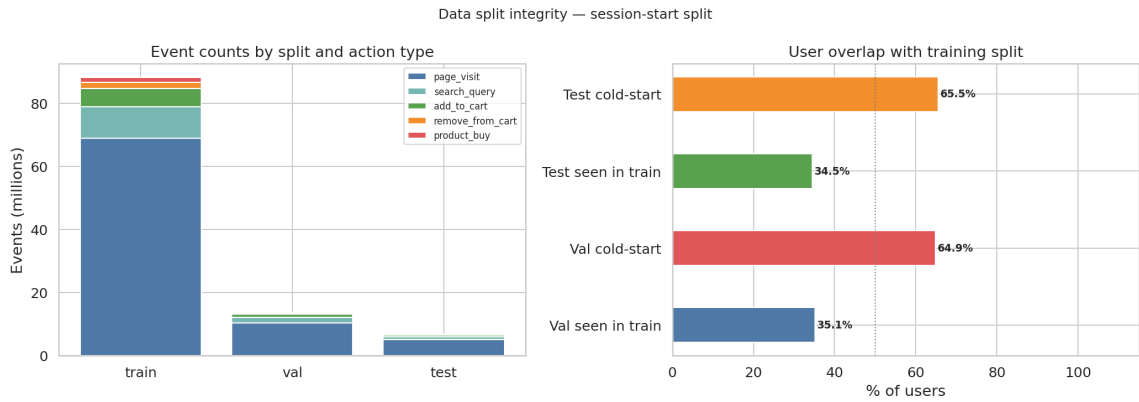


Fig. 9. Data split integrity under the session-start split. Left: stacked event counts per split, broken down by action type. Right: user overlap with the training split. Roughly two-thirds of validation and test users have no training history and are cold-start.

pipeline explicitly under-represents the long tail in exchange for a tractable embedding table; results in the tail are not directly addressable by the generator.

Strong temporal item drift. Relative to the train top- K vocabulary, event-level OOV nearly doubles from training to test (Table 5): the catalog itself shifts over the held-out window, so absolute test metrics conflate generation quality with item-vocabulary drift. This cannot be eliminated without re-training on a moving window.

Cold-start dominates the held-out splits. Approximately two-thirds of validation and test users do not appear in training (Figure 9), so user-conditioned generation is evaluated mostly on users without historical context, and personalization is upper-bounded by population-level priors for them.

Soft rather than hard behavioral constraints. Only 53.3% of training purchases follow a same-session same-SKU add_to_cart, and 36.5% have no prior same-SKU cart event of any kind (Figure 8); a hard prerequisite would mark

over a third of valid purchases as invalid. The pipeline therefore combines hard validity checks (the bigram structure) with softer behavioral constraints rather than one rigid rule.

Item names are opaque numeric vectors, not text. The name field is a 16-byte numeric vector, near-uniformly distributed (99.9% of maximal entropy) yet carrying real category structure (nearest-neighbour same-category rate 19.71% versus 0.25% chance, a $78.85\times$ lift): a quantized pre-computed embedding, not random noise or tokenizable text. The pipeline uses it as a learned input feature embedding (Section 6.3) but cannot deploy text-encoder-based item heads.

Domain and provenance are retail-specific. The dataset is a single-vendor (Synerise) retail log, with two scope consequences: the extracted rules and validity checks are e-commerce conventions that do not transfer automatically to other domains, and the data comes from one tracking platform serving particular merchants and audiences, so demographic, geographic, and device-mix biases are present but not observable in the schema. Provenance and anonymization are covered below.

5.4 Data Handling and Governance

Anonymization is provider-supplied (no formal k -anonymity or differential-privacy guarantee), and the legal basis and PII treatment are detailed in Section 4; the thesis treats `client_id` and `sku` as opaque keys and makes no attempt to re-identify, link, or invert them. The schema contains no free-text content, contact information, or precise geolocation. All reported analyses are population-level; intermediate artefacts are kept locally and not redistributed, and published aggregates, computed over millions of events and hundreds of thousands of users, have no cell small enough to single out an individual.

6 Methodology: Model and Architecture

6.1 Interaction Schema and Interface Contract

The minimum generated event tuple is defined in Table 6. This schema formalizes what “interaction” means in the simulation context and determines what each prediction head of the generator must produce.

Table 6. Output log schema. Fields marked *generated* are produced directly by the generator; *derived* fields are computed post hoc.

Field	Type	Description
<code>user_id</code>	integer	<i>Generated by arrival process.</i> Identifies the synthetic user.
<code>session_id</code>	integer	<i>Generated by arrival process.</i> Groups events into one session.
<code>action_type</code>	categorical	<i>Generated by transformer model.</i> One of: <code>page_visit</code> , <code>search_query</code> , <code>add_to_cart</code> , <code>remove_from_cart</code> , <code>product_buy</code> .
<code>item_id</code>	integer	<i>Generated by transformer model.</i> The item involved in the event. Null for <code>search_query</code> events, which do not target a specific item.
<code>delta_t</code>	float	<i>Generated by transformer model.</i> Inter-event time in seconds.
<code>event_time</code>	timestamp	<i>Derived.</i> Cumulative <code>delta_t</code> from session start.

The `item_id` field is nullable: the item head is suppressed for `search_query` and `page_visit` events, which carry no item information in the dataset. Session boundaries are determined by a generator-predicted EOS token, subject

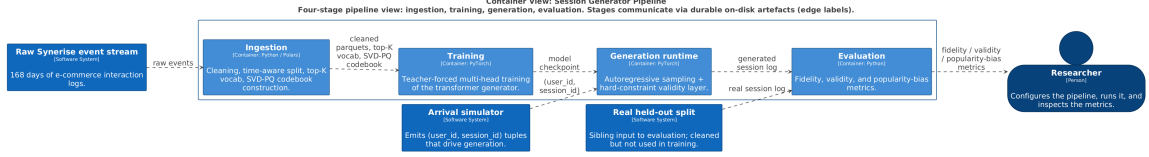


Fig. 10. Container-level view of the system (C4 container diagram) [6]. Stages communicate through durable artefacts (edge labels) rather than in-memory handoffs, so any single stage can be re-run with its inputs held fixed. Model internals are detailed in Section 6.3; the generation-time component view is in Figure 15 (Appendix A).

to a maximum-length cap. The generator accepts the arrival simulator’s output as input and produces the schema of Table 6; context features (category, price) are model inputs but excluded from the output schema.

6.2 System Architecture

The pipeline runs in four stages: *ingestion*, *training*, *generation*, and *evaluation*, each an independent process communicating through durable on-disk artefacts, so any stage can be re-run with its upstream artefacts held fixed. Figure 10 gives the container-level view; model internals are deferred to Section 6.3 and the validity layer to Section 6.4. The four-stage decomposition extends the simulator architecture of supervisor Diptish Dey (CMI lab).

Stage 1: Ingestion. The raw Synerise event stream is cleaned, time-aware split, and reduced to a top- K item vocabulary mapping the catalog tail to a shared zero index. A second offline job builds the SVD-PQ codebook [34] used by Iteration 3 (truncated SVD of the weighted user–item matrix, then per-dimension quantile binning, one token tuple per SKU; Sections 5 and 6.3.3).

Stage 2: Training. The loader emits (prefix, target) pairs plus a cross-session memory window of the user’s H most recent prior events. The transformer is trained with teacher forcing and a summed cross-entropy loss over the three factored heads[41]; the action head’s EOS class is the final target of every session, so termination is learned from real boundaries rather than a fixed cap. The stage outputs one checkpoint (Sections 6.3 and 6.6).

Stage 3: Generation runtime. The arrival simulator yields (user_id, session_id) tuples driving a session-based autoregressive generator: the checkpoint is unrolled step by step and the three heads jointly emit an action, item, and inter-event time until EOS, with the user’s cross-session memory in the input (Section 6.3). Each session passes through the hard-constraint validity layer (Section 6.4) before being written, reported pre- and post-repair so enforcement cost is measurable. The component view is Figure 15 (Appendix A).

Stage 4: Evaluation. The stage consumes the generated log and the real held-out split as sibling inputs and produces the fidelity, validity, and popularity-bias metrics of Section 7. Both inputs are plain logs sharing the schema of Table 6, so the same harness evaluates the transformer and the Markov baseline (Section 6.5), the prerequisite for a fair test.

Interfaces and artefacts. Five artefacts pin the system down: the cleaned and split parquets, the top- K vocabulary, the SVD-PQ codebook [34], the trained checkpoint, and the generated session log (Table 6). Each has a stable on-disk representation, so the four stages can be developed and re-run independently as long as the upstream artefact is fixed.

6.3 Model Architecture

The project evaluates three model architectures across successive iterations. All three share the same transformer backbone, action head, temporal head, and cross-session memory, and differ only in how the next SKU is predicted from the shared hidden state (Figures 16–18, Appendix A). Each iteration is trained and evaluated identically on the metric suite of Section 7, and each iteration’s measured failure mode motivates the next.

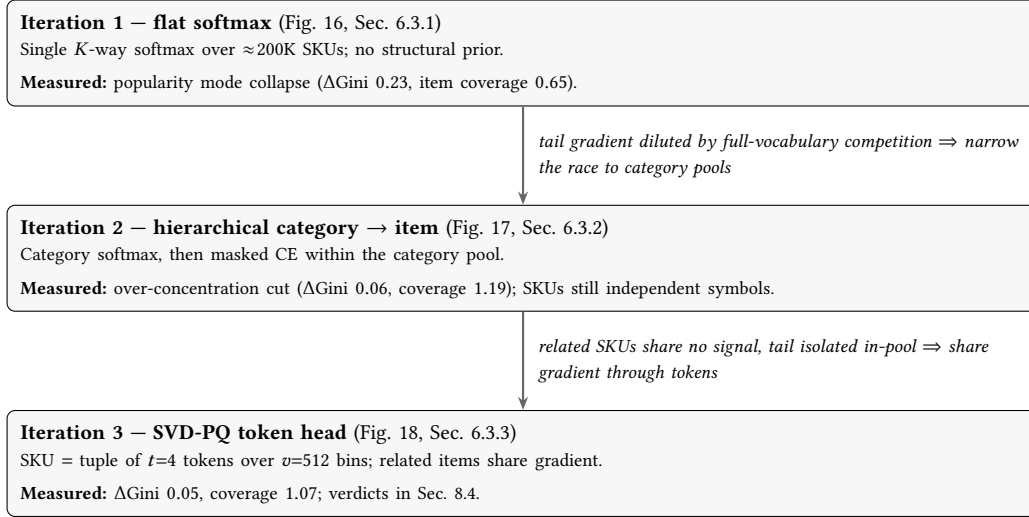


Fig. 11. The iteration loop: every iteration is trained identically and scored on the full metric suite, and the measured failure mode of one iteration is the design input of the next (arrow labels). ΔGini and coverage are train-reference values from Section 8.1.

Shared scaffold. At every position the input is the elementwise sum of d -dimensional embeddings of action type, item, inter-event time, position, and the auxiliary item-side attributes (category, price, name). A causal transformer decoder[40] attends over the running session prefix together with a cross-attention memory built from the user’s prior sessions (window of H events) and emits a hidden state at each step. Three factored heads share this hidden state: an action head over the five action types plus an end-of-session class, an item head whose form is the subject of the iterations below, and a temporal head over discretized log-spaced inter-event-time bins. Training is teacher-forced with summed cross-entropy losses on the three heads (simulation/generator/session_transformer.py).

Cross-session memory window. The window size $H = 16$ matches the mean session length on the cleaned training corpus ($|\overline{s}| = 15.97$ events), so on average one full prior session of context is available at each step: the cross-attention budget set at the natural unit of behaviour, a session. Under this choice the entire prior history of 60.9% of training users fits losslessly inside the window; the rest contribute their most recent H events. Cost doubles each time H doubles while marginal coverage shrinks (Table 7), so $H = 16$ sits on the cheap side of the coverage–cost knee, with $H = 32$ the natural ablation.

6.3.1 Iteration 1: Flat softmax over the full SKU vocabulary. The reference iteration predicts the next SKU directly with a single K -way softmax ($K \approx 200,000$) conditioned only on the sampled action (Figure 16): vanilla cross-entropy over the global SKU index, with no masking, factorization, or offline preprocessing. Its purpose is methodological: establish

Table 7. Cross-session memory window coverage. “Lossless coverage” is the share of training users whose entire event history fits inside the window; cross-attention cost per decoder layer scales linearly in H (right column: relative to $H = 16$).

H	Lossless coverage	Cross-attn cost (rel.)
16	60.9%	1×
32	79.2%	2×
64	90.5%	4×
128	96.2%	8×
256	98.7%	16×

whether the simplest formulation already yields acceptable fidelity, validity, and bias, so later iterations improve on a real reference rather than a strawman. Its two structural weaknesses set the agenda: tail-SKU gradient is diluted by softmax competition with the popular head of the catalog, surfacing as the measured popularity over-concentration (Section 8.1) that Iteration 2 targets; and SKUs are independent symbols with no notion of substitutability, which motivates Iteration 3.

6.3.2 Iteration 2: Hierarchical category $\rightarrow$ item. Motivated by Iteration 1’s failure: the popular head of the catalog dominates the K -way gradient and the tail collapses. Iteration 2 factorizes the flat softmax into a two-level class $\rightarrow$ member decision, the standard remedy for an intractably large output vocabulary [32]: a CategoryHead (Figure 17) first picks a category, and the item head samples a SKU within that category’s pool, implemented as a vectorized masked cross-entropy at training time and a sliced, renormalized weight matrix at inference (see the *Ontwikkeling* section of the repository wiki [30] for implementation details). Tail items now compete only against their category siblings rather than the entire popularity-dominated vocabulary, targeting Iteration 1’s over-concentration without touching the action or temporal heads. The factorization narrows the competition rather than removing it: popular SKUs still dominate within their pools and the category softmax inherits the cross-category skew, so over-concentration is expected to shrink rather than vanish; the residual is quantified in Sections 8.1 and 8.3.

6.3.3 Iteration 3: SVD-PQ token-factored item head. Motivated by the weakness Iteration 2 leaves unaddressed: SKUs are still independent symbols, so related items share no gradient and tail items remain isolated even within a category pool. Iteration 3 replaces the per-SKU softmax with t shared discrete tokens from an offline SVD-PQ codebook [20, 34], so semantically related SKUs share leading tokens and therefore gradient signal (Figure 18). Each SKU is encoded offline as a tuple of $t = 4$ tokens over $v = 512$ bins via truncated SVD of the weighted user–item interaction matrix followed by per-dimension quantile binning (`ingestion/svdpq.py`). This spreads item-side gradient across tv token cells rather than K classes and collapses head parameters from $O(Kd)$ to $O(tvd)$ ($\approx 200,000d \rightarrow 2,048d$). Following GPTRec [34], shared leading tokens are expected to mitigate Iteration 1’s mode-collapse. Model selection follows validation loss, as in the earlier iterations. The pre-registered pass condition, bias group on par with or better than Iteration 2 *and* TSTR Hit Rate@10 (HR@10) above Iteration 2 (the utility axis later archived, Appendix B.1), is resolved in Section 8.4.

Event-type weighting. Each (*user*, *item*) cell of the interaction matrix sums per-event inverse-frequency weights $w_a = \log(N/n_a)$ (Table 8), so rare preference-expressing actions outweigh abundant page-visit traffic in the SVD factors; the weighted matrix receives BM25-style damping and per-user L_2 normalization before factorization, so power users and already-popular items do not dominate the factors (see the *Ontwikkeling* section of the repository wiki [30] for implementation details).

Table 8. Per-action SVD-PQ event weights $w_a = \log(N/n_a)$ computed on the cleaned event corpus ($N = 101,859,688$ events, test split excluded).

Action type a	n_a	w_a
page_visit	79,222,116	0.25
search_query	11,649,993	2.17
add_to_cart	6,855,719	2.70
remove_from_cart	2,325,119	3.78
product_buy	1,806,741	4.03

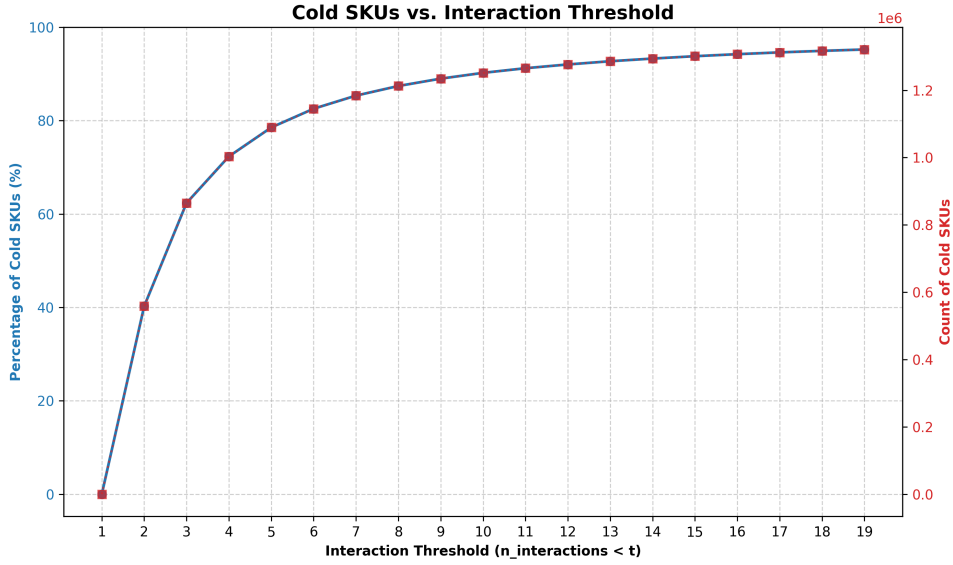


Fig. 12. Share (left axis) and absolute count (right axis) of SKUs classified as cold under varying interaction thresholds $\tau \in \{1, \dots, 19\}$ (cold $\Leftrightarrow n_interactions < \tau$). The pipeline uses $\tau = 2$, the smallest value at which every SVD-embedded SKU has support from at least two distinct users. The curve is heavy-tailed: roughly 40% of SKUs appear once and would be unembeddable by SVD regardless of threshold choice.

The item head (SVD PQ ItemHead) is a single linear projection reshaped into t parallel v -way logit vectors, trained with mean cross-entropy over the t token positions under label smoothing ($\epsilon = 0.1$) [33, 39]. Because the head scores a SKU by the product of its t token probabilities, the smoothing bounds any single SKU’s joint probability near 0.66 at the training optimum, an explicit guard against the winner-take-all concentration of Iteration 1.

Cold-SKU fallback. SKUs below `svdpq_min_interactions=2` are assigned their category centroid in token space: a SKU seen in one user–item cell carries no co-occurrence signal, so 2 is the minimum at which every SVD-embedded SKU has support from two distinct user contexts. $\approx 40\%$ of catalog SKUs fall back, but these are by construction the rarest and a tiny share of training item-bearing events; raising the threshold to 5 would push $\approx 78\%$ into fallback while discarding usable signal (Figure 12).

6.4 Hard-Constraint Validity Layer

Generated sessions pass through explicit validity checks before output:

- (1) **Invalid action transitions:** action bigrams not observed in training data are flagged and counted.
- (2) **Timestamp monotonicity:** events where $\delta_t \leq 0$ are flagged.
- (3) **Purchase-without-prior-exposure:** a `product_buy` of a SKU not currently in the session’s cart (added by `add_to_cart` and not removed since) is flagged.

Both pre-constraint and post-constraint statistics are reported, making the cost of constraint repair independently measurable.

Design safeguards. The validity layer is the AI-specific guardrail against behaviorally implausible outputs, with pre/post-repair reporting turning its decisions into an auditable signal rather than a silent rewrite. The cold-SKU fallback (Section 6.3.3) complements it as a graceful-degradation path at the long tail.

6.5 Comparison Portfolio

The portfolio separates two comparison axes: fidelity and validity compare the transformer against a baseline generator on identical metric sets, while the popularity-bias axis compares the three transformer iterations against each other and a real-continuation reference (§7.4).

Compared generators (fidelity and validity).

- **Markov generator.** A first-order Markov chain over action types, with item selection drawn independently from the empirical popularity distribution of the training split. The Markov chain has no action–item coherence by construction: the sampled item does not condition on the sampled action at the same step. This generator mirrors the interaction generator currently in use within the CMI simulator (§2.4), so the comparison is directly relevant to this thesis’s improvement objective.
- **Transformer generator.** The proposed multi-head autoregressive transformer described in §6.3, whose factored heads emit action, item, and inter-event time jointly and are therefore coherent by construction.

Reference point (popularity bias). The bias axis requires per-user conditional generation, which the popularity-sampled Markov chain cannot produce; it therefore compares the three transformer iterations only. Its reference is not a trained model but *real behaviour*: for the same users and conditioning point, the items each user actually interacted with next (§7.4). The reference establishes the organic level of popularity drift; a faithful generator should approach it, and exceeding it quantifies amplification.

What is held constant. Both generators are scored on identical metric sets against the same references. On the bias axis, the basket protocol (user sample, conditioning histories, basket size, seeds) is identical across iterations: the same seed selects the same users for every model, so cross-iteration differences are paired rather than confounded by user sampling.

Scope of the resulting claim. The claim supported by this portfolio is narrow: *a constraint-aware multi-head transformer outperforms a popularity-sampled first-order Markov baseline on the Synerise dataset on session-shape fidelity and validity, and its popularity-bias amplification is quantified per iteration against real user behaviour.* It is not a general claim that transformers dominate Markov-chain generators; that would require GRU- or GAN-based baselines and cross-dataset

replication, both scoped to future work (§9). The portfolio is sized to support a defensible improvement claim against the specific baseline this thesis is positioned to replace.

6.6 Hyperparameter Tuning and Training Procedure

The pipeline is configuration-driven: every tunable knob is exposed through a single YAML file (`config.yaml`) and consumed by all four stages without code changes. The values used are listed in Appendix C and in the repository [31].

7 Methodology: Evaluation

7.1 Evaluation Setup

Evaluation covers three dimensions: fidelity (do generated sessions look like real ones?), validity (do they respect behavioral constraints?), and popularity bias (do generated interactions over-represent popular items relative to real behaviour, and differentially across user groups?). All metrics are reported across five random seeds as mean and standard deviation, against reference distributions precomputed once from the training split. The design is grounded in the criteria of Section 7.5.

7.2 Fidelity Metrics

The fidelity layer probes distinct failure modes rather than reporting a single distance: each metric targets a separable property, a generator can pass one and fail another, and the combination is what supports a fidelity claim. The full set, with ranges and rationale, is in Table 14 (Appendix B).

Five distributional metrics compare generated sessions against the real training distribution, two behavioral-rate metrics check funnel mechanics, and a bias group tracks item-side mode collapse without entering the primary fidelity claim [17, 22]. Bias metrics are computed against a size-matched random subsample of the real split; without this, item coverage and Gini are mechanically depressed for any batch smaller than the multi-million-session reference. Item coverage is additionally reported only against the training reference: the generator emits training-vocabulary items only, so a validation-pool denominator would measure catalogue churn rather than fidelity. Both generators are evaluated on the identical metric set, making the transformer–Markov comparison direct.

7.3 Validity Metrics

Generated sessions are scored on the three constraints of the validity layer (Section 6.4), each reported as the fraction of violating sessions before and after repair, making the repair cost visible (`evaluation/validity_checker.py`).

7.4 Popularity-Bias Metrics

The bias axis adapts the four group-level popularity-bias metrics of Braun et al. [5] to per-user recommendation baskets generated by each model iteration (`generate_baskets.py`, `evaluation/popularity_bias.py`). Both the capability and the metric choice originate from the stakeholder loop of Section 3: per-user basket generation was requested by CMI researcher Debarati Bhaumik, and the metric suite is the CMI group’s own published instrument.

Basket protocol. Per iteration, 10,000 unique real training users with at least two sessions are sampled; each is anchored on their latest training session containing an in-vocabulary item event, and the model is conditioned on the user’s prior-session history plus that session’s seed event. The same seed selects the same users for every model (seeds 42–46), so cross-iteration comparisons are paired. Baskets are read under two lenses: *top-k head* ($k=20$ most

probable SKUs conditioned on `add_to_cart`; the deployment view) and *sampled sessions* (a free-running rollout filtered to positive item events, deduplicated, capped at 20; the data-product view).

User groups. Synerise carries no demographic attributes, so the sensitive groups of Braun et al. are replaced by popularity-affinity groups [1]: all 2.25M training users are median-split on the mean popularity of their training profile into *niche* and *mainstream* users. Item popularity $\phi_i = N_i/N_U$ is the fraction of training users that interacted with item i ; a user’s profile is their distinct positive (user, item) pairs before the training cutoff.

Metrics. Each basket set is scored per user group on four metrics. (1) The *Gini coefficient*: how concentrated are the recommended items, from 0 (every item equally often) to 1 (one item dominates). (2) ΔGAP : how much more popular are the generated items than the user’s own history. The group average popularity (GAP) is the mean popularity ϕ of a user’s basket (r) or profile (p) items, averaged over the group, and $\Delta\text{GAP} = (\text{GAP}_r - \text{GAP}_p)/\text{GAP}_p$: 0 is calibrated to history, 1 is twice as popular. (3) The *between-group* variant $\Delta\text{GAP}_{\text{revised}} = (1 - \text{GAP}_r)/(1 - \text{GAP}_p)$, near-degenerate at e-commerce popularity scales ($\phi \ll 1$) and therefore reported but not interpreted. (4) The *cosine similarity* between the two groups’ item frequencies: do niche and mainstream users receive different items (0) or the same items (1)? Braun et al. measure bias dynamics over a simulated feedback loop; here each basket set is one feedback step, and the dynamic reading comes from comparing model iterations.

Real-continuation reference. The reference replaces a model-based ceiling with real behaviour (`generate_reference_baskets.py`): for the same anchor users, the “basket” is the anchor session’s actual positive in-vocabulary items *after* the seed event, deduplicated and capped identically. This measures the organic level of popularity drift; a faithful generator should approach that level rather than zero, and the gap above it quantifies amplification. Gini, coverage, and cosine are basket-size-sensitive, so comparisons are made within a lens only, and the real reference (mean 1.6 items per user) is compared against the size-matched sampled-sessions lens (mean 1.7–2.1).

7.5 Reliability and Validity

Reliability. Test–retest reliability is operationalized through five-seed variance reporting: five seeded runs play the role of repeated measurements, and low variance indicates the measurement is stable rather than an initialization artefact, following the repeated-run practice of T-RECS [26].

Construct validity. The fidelity metrics are non-redundant: the action Jensen–Shannon divergence (JSD) captures the marginal behavioral mix [24]; the action-bigram L1 checks sequential ordering (a generator can pass JSD and fail bigram L1); KS on session length and on δ_t separate wrong session shape from wrong pacing; the conversion and cart-abandonment deltas anchor the funnel mechanics; and the sample-diversity ratio guards against near-duplicate sessions behind matching marginals. The fidelity-side bias group is kept out of the primary fidelity claim because it diagnoses the item-side pathology that Iterations 2 and 3 explicitly target, and so is a development signal; the group-level basket metrics of Section 7.4 are the primary bias axis, anchored to the real-continuation reference rather than a distributional match.

Ecological validity. Offline evaluation results are bound to static log fidelity, validity rates, and the popularity-bias profile of generated sessions; they do not support claims about generator performance under live deployment with real feedback dynamics.

Content validity. The five-action vocabulary covers the complete observed action space of the Synerise dataset. No action type is excluded from the evaluation.

8 Results

8.1 Fidelity

Do the transformer iterations reproduce the real distribution better than the Markov baseline? On session shape and the bias group, yes; on the overall action-type mix Markov is unbeatable by construction. The reason is structural: Markov is fit directly to that mix but has no session-length or sequential model, so it cannot reproduce realistic session shape. The transformers invert this profile. The supporting metrics, with their ideal directions, are in Tables 9 (test split) and 10 (training split): five-seed means over 500,000 generated sessions, against an identical Markov baseline and a Train-Real-Test-Real (TRTR) floor: real data scored against the same reference.

Table 9. Test-split fidelity, five-seed means. Unmarked metrics are discrepancies against the real reference: lower is better, 0 is perfect, and the TRTR column shows the real-vs-real floor a perfect generator would approach. KS is the largest distance between the real and generated cumulative distributions: 0 = identical, 1 = disjoint. §: no better/worse direction; the target is the TRTR value. Seed std ≤ 0.003 except unique-SKU counts (std ≤ 240).

Metric	Iter 1	Iter 2	Iter 3	Markov	TRTR
<i>Distributional</i>					
JSD(action)	0.0066	0.0051	0.0054	0.0005	0.0005
KS(session length)	0.114	0.186	0.104	0.946	0.040
KS(δ_t)	0.251	0.251	0.251	0.251	0.041
L1(action bigrams)	0.195	0.149	0.181	0.054	0.055
Sample diversity [§]	1.000	1.000	1.000	1.000	1.000
<i>Behavioral rate</i>					
Δ conversion rate	0.107	0.133	0.133	0.132	0.009
Δ cart abandonment	0.030	0.112	0.005	0.238	0.024
<i>Bias (informational)</i>					
Unique synth. SKUs [§]	74,258	135,324	122,176	155,814	125,522
Popularity JSD	0.329	0.312	0.322	0.453	0.295
Δ Gini	0.147	0.027	0.035	0.336	0.087

Item coverage is intrinsic to the generated catalogue and reported once, against the training reference (Table 10): the generator can emit only training-vocabulary items, so a test-pool denominator measures two-week catalogue churn ($\approx 68k$ vs. $\approx 114k$ distinct SKUs per matched sample) rather than fidelity; the real-vs-real sample itself scores 1.7 on it. Generalization to the unseen window is carried by the JSD, Δ Gini, and popularity-JSD rows, reported on both splits.

Table 10. Training-split fidelity (generalization reference), five-seed means. Unmarked metrics: lower is better, TRTR column = real-vs-real floor. †: ratio with ideal 1.0 (< 1 under-covers, > 1 over-covers). Seed-std conventions as in Table 9.

Metric	Iter 1	Iter 2	Iter 3	Markov	TRTR
<i>Distributional</i>					
JSD(action)	0.0074	0.0042	0.0044	0.0002	0.0000
KS(session length)	0.137	0.152	0.072	0.938	0.001
KS(δ_t)	0.283	0.283	0.283	0.283	0.002
L1(action bigrams)	0.177	0.117	0.152	0.018	0.001
<i>Bias (informational)</i>					
Item coverage [†]	0.647	1.185	1.071	1.338	0.998
Popularity JSD	0.211	0.185	0.195	0.345	0.142
Δ Gini	0.232	0.059	0.051	0.250	0.001

Markov vs. transformer split as expected. Markov is near-optimal on metrics it hits by construction (action JSD 0.0005, action-bigram L1 0.054) but fails on session shape (KS 0.946 vs. 0.104–0.186: a near-total mismatch against a close match), on both splits.

Action and temporal fidelity are close across iterations. Action JSD and action-bigram L1 are lowest for Iteration 2 (0.0051, 0.149). Session shape is non-monotonic: KS(session length) moves $0.114 \rightarrow 0.186 \rightarrow 0.104$ across Iterations 1–3, with Iteration 3 closest to the TRTR floor (0.040). $KS(\delta_t)$ is identical across all generators because all sample from the same time bins (Section 9.2).

The bias group is a trade-off, not a clean ranking. Lower Δ Gini and popularity-JSD mean a closer match to real popularity inequality; item coverage near 1.0 means the right number of distinct SKUs relative to the training reference. Iteration 1 under-covers (0.647) and concentrates worst (Δ Gini 0.232/0.147 train/test, popularity-JSD 0.211/0.329); Iterations 2 and 3 invert this profile, matching popularity inequality far better (Δ Gini 0.059/0.051 train) while mildly over-covering (1.19/1.07); Markov over-covers most (1.34). Popularity-JSD stays above the TRTR floor for all three. The structural priors [34] thus shift the failure mode from over-concentration toward mild over-coverage rather than removing it.

8.2 Validity

Are generated sessions structurally legal? Yes, the two hard constraints are never violated, and the one soft constraint is fully repairable at low cost.

Table 11. Pre- and post-repair validity rates, five-seed means. Lower is better. Seed std ≤ 0.001 throughout.

Constraint	Pre-repair			Post-repair		
	Iter 1	Iter 2	Iter 3	Iter 1	Iter 2	Iter 3
Invalid action transition rate	0.000	0.000	0.000	0.000	0.000	0.000
Timestamp monotonicity violations	0.000	0.000	0.000	0.000	0.000	0.000
Purchase-without-prior-exposure	0.156	0.177	0.177	0.000	0.000	0.000

All three iterations satisfy the action-transition and timestamp-monotonicity constraints at generation time (0.000 pre- and post-repair, all seeds). Only purchase-without-prior-exposure needs repair: 15.6%/17.7%/17.7% pre-repair, 0.000 after, stable across seeds and below the 36.5% rate at which real purchases lack a prior same-SKU cart event, so all three generators are stricter about cart-before-purchase ordering than the real data.

8.3 Popularity Bias

Is the generated traffic’s popularity bias calibrated to the real level? For mainstream users, yes once the structural priors of Iterations 2–3 are in place; for niche users, no: every iteration delivers items $\approx 13\times$ more popular than those users’ own histories, where the real continuation drifts only mildly upward (Δ GAP 0.45), and no architectural change moved that number. Each iteration generates baskets for the same 10,000 anchored users per seed (Section 7.4), read under two lenses: *sampled sessions* (the data product, anchored by the real-continuation reference) and the *top-k head* (the deployment view). Table 12 reports the primary lens; both lenses and the reference are visualized in Figures 13 and 14.

Mainstream-user popularity is calibrated by the structural priors. Iteration 1 amplifies mainstream-profile popularity (Δ GAP 0.26) to twice the organic level (0.13); Iterations 2 and 3 land at -0.06 and 0.06 , at or slightly below real drift.

Table 12. Popularity bias on sampled sessions, five-seed means $\pm$ std. Δ GAP: relative popularity of generated items against the group’s own profiles (0 = calibrated to history). Coverage: distinct recommended SKUs over the 1.24M-item catalog. For every row, the real column is the value a faithful generator should match.

Metric (group)	Iter 1	Iter 2	Iter 3	Real
Δ GAP (mainstr.)	$0.26 \pm .03$	$-0.06 \pm .02$	$0.06 \pm .04$	$0.13 \pm .03$
Δ GAP (niche)	13.2 ± 1.3	12.6 ± 0.5	13.3 ± 1.1	$0.45 \pm .04$
Gini (mainstr.)	0.153	0.127	0.113	0.141
Gini (niche)	0.046	0.051	0.043	0.005
Coverage (mainstr.)	0.32%	0.49%	0.37%	0.36%
Coverage (niche)	0.16%	0.23%	0.17%	0.14%
Cosine niche $\leftrightarrow$ mainstr.	0.204	0.195	0.166	0.011

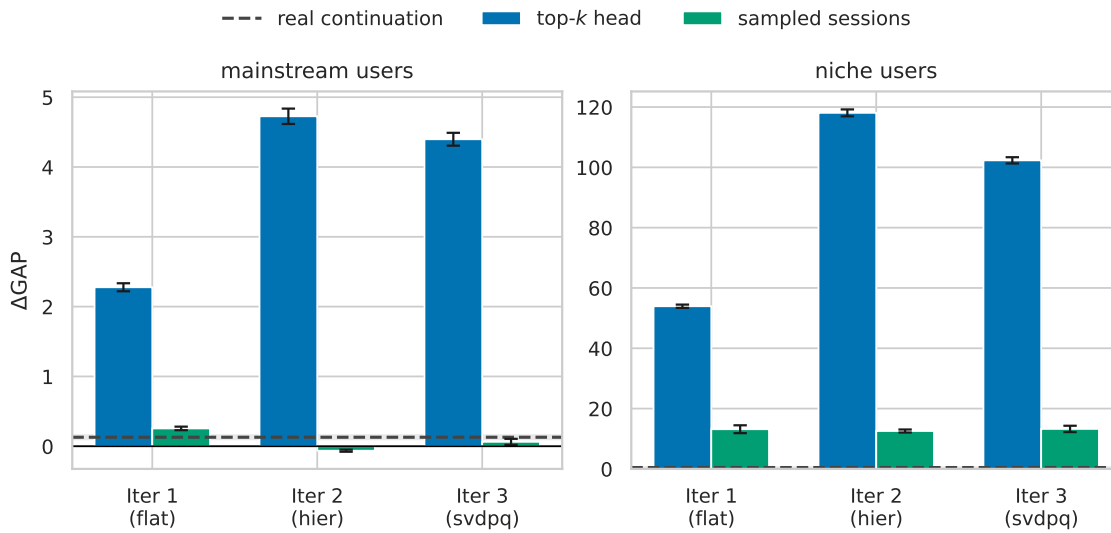


Fig. 13. Δ GAP per user group and lens, five-seed means. The dashed band is the real-continuation reference; note that in the niche panel it sits at +0.45, not zero, the 0–120 axis only makes it appear flat against the generators’ $\approx 13\times$ amplification.

Iteration 2’s mild negative reads together with its over-coverage (Table 10): it samples the tail slightly more than real mainstream behaviour does.

Niche-user amplification is invariant to architecture. All three iterations deliver items $\approx 13\times$ more popular than niche users’ own histories (12.6–13.3), against an organic drift of +0.45: roughly $29\times$ the real level, unchanged by any item-head redesign. The iterations repaired mainstream calibration but never touched the group popularity bias harms most.

Generated groups are homogenized. Real niche and mainstream continuations are nearly disjoint (cosine 0.011); every iteration produces overlapping distributions (0.166–0.204 sampled, 0.96–0.99 at the head), recommending largely the same popular items regardless of user history.

The two lenses locate each architecture’s bias. At the top- k head the ranking inverts: Iterations 2/3 concentrate hardest (a single SKU appears in 52–58% of users’ top-20, mainstream Δ GAP 4.7/4.4, cosine 0.99) while Iteration 1’s head is the

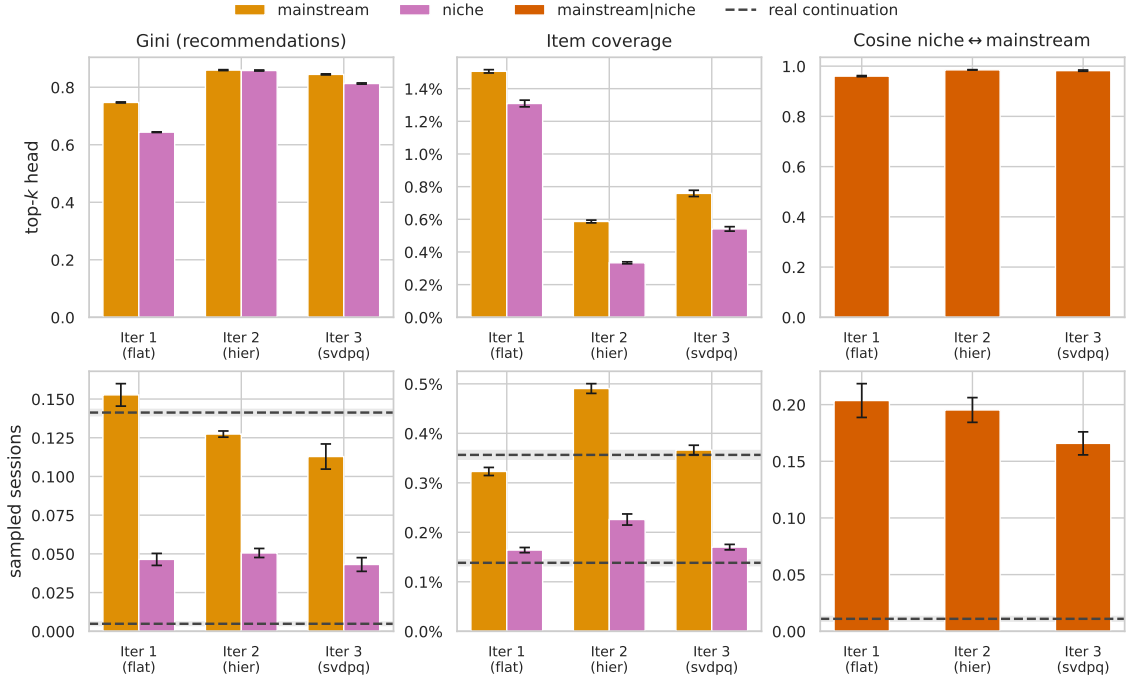


Fig. 14. Recommendation concentration and group homogenization per lens. Gini, coverage, and cosine are basket-size-sensitive, so values are comparable within a row (lens) only; the real reference is drawn on the size-matched sampled-sessions row.

most user-differentiated (36%, ΔGAP 2.3). Iteration 1’s failure lives in its sampled distribution; Iterations 2/3 invert the profile. The sampled-sessions ranking agrees with the fidelity chapter’s concentration ordering (convergent evidence across independent metric families), whereas the archived TSTR evaluation (Appendix B.1) inverted it by rewarding exactly the concentration the bias metrics expose.

8.4 Pre-registered pass conditions

Table 13. Status of the pre-registered pass conditions of Section 6.3 on the five-seed data.

Condition	Status
Iter 2: action/temporal not regressed vs Iter 1	<i>Mixed</i> (action/ δ_t improved; KS session length $\uparrow$)
Iter 2: coverage and ΔGini improved vs Iter 1	<i>Met</i> (coverage $0.65 \rightarrow 1.19$, ΔGini $0.23 \rightarrow 0.06$)
Iter 3: action/temporal not regressed vs Iter 2	<i>Mixed</i> (session length $\downarrow$; action bigram $\uparrow$)
Iter 3: bias group $\geq$ Iter 2	<i>Mixed</i> (coverage $\downarrow$ toward 1.0; popJSD/ ΔGini $\uparrow$ slightly)
Iter 3: TSTR > Iter 2	<i>Not met</i> (indistinguishable; vocabulary-confounded, Appendix B.1)
Cross-arch: transformer > Markov	<i>Met</i> (session-shape and TSTR)
Iter 1 (flat softmax): mode-collapse exhibited	<i>Confirmed</i> (TSTR-T > TRTR; worst ΔGini /popJSD)

Several conditions returned *Mixed* or *Not-met*, so the pre-registered conditions acted as a falsification mechanism rather than a rubber stamp: the Iteration 3 > Iteration 2 TSTR condition failed (indistinguishable, vocabulary-confounded), the

action/temporal no-regression conditions are only partially met (non-monotonic session shape), and Iteration 1’s mode-collapse is confirmed by its synthetic-above-real TSTR. The cross-architecture condition is the one unambiguous TSTR outcome: a GRU4Rec trained on transformer-generated sessions and evaluated on real test data reaches 0.124–0.253 HR@10 against 0.008–0.009 when trained on Markov-generated sessions, a $\approx 14\text{--}30\times$ gap (Appendix B.1). The utility axis was subsequently replaced by the popularity-bias evaluation (Section 8.3): TSTR rewarding Iteration 1’s mode collapse is the limitation pre-registered in Section 9.4 materializing in the data.

9 Discussion

9.1 Interpretation of Findings

The five-seed picture is a coverage-versus-concentration trade-off, with each iteration’s popularity-amplification profile decomposed by the basket-level bias metrics. Mapped onto the four sub-questions:

- **SQ1 (formal interaction definition).** The two-level definition, typed event tuples grouped into sessions with cross-session user state, held under generation: the schema of Table 6 was emitted with zero hard-constraint violations, and the only soft prerequisite needing repair (15.6–17.7%, below the 36.5% real-data rate) was bounded and seed-stable. The definition was expressive enough to generate against directly, not only to validate post hoc.
- **SQ2 (required architectural properties).** The properties SQ2 names separate the transformer from the Markov baseline: the factored heads gave a wide margin on session shape (KS 0.10–0.19 vs. 0.95) where the coherence-free baseline fails. The structural priors of Iterations 2–3 cut Iteration 1’s over-concentration (ΔGini 0.23 $\rightarrow$ 0.05–0.06 train) and calibrated mainstream-user popularity to the real-continuation level, yet left niche-user amplification untouched ($\approx 13\times$ vs. organic 0.45): coherence and sequence capacity are confirmed necessary, while bias-faithfulness needs more than architecture alone.
- **SQ3 (feasible implementations and baseline).** The chosen scope (three transformer iterations against a popularity-sampled Markov baseline mirroring the deployed CMI mechanism) was executable within the timeline and supported a like-for-like comparison: identical metric sets against the same references, and a paired basket protocol on the bias axis (Section 6.5). The baseline cannot produce per-user conditional baskets, so it does not participate in the bias axis; a stronger generator baseline is future work (see Limitations).
- **SQ4 (falsifiable, reproducible protocol).** The pre-registered pass conditions made “better than baseline” falsifiable: several returned Mixed or Not-met (Table 13), and the small five-seed variance makes the differences that exceed it reproducible rather than seed artefacts.

Taken together, the sub-question answers resolve the main question’s *to what extent*: the generator produces logs with full structural validity (zero hard-constraint violations), session-shape fidelity far beyond the Markov baseline (KS 0.10–0.19 vs. 0.95), and popularity bias calibrated to the real-continuation reference for mainstream users, but not for niche users, whose $\approx 13\times$ amplification no architectural iteration removed.

9.2 Validity

Internal validity. The main safeguard is the five-seed protocol (Section 7.1); the variance magnitudes (Section 9.3) are small enough that the cross-iteration differences in this chapter are not seed artefacts. Two anomalies are flagged. First, $\text{KS}(\delta_t)$ is identical to four decimals across all iterations and Markov (0.2506 test, 0.2833 train, std 0.0000), most plausibly because every generator samples δ_t from the same log-spaced bins, so all sit the same distance from the continuous real distribution. Second, $\Delta\text{GAP}_{\text{revised}}$ and the between-group GAP are near-degenerate at this catalog’s popularity

scale (Section 7.4), a measurement limit rather than evidence of between-group fairness; the archived TSTR anomaly is documented in Appendix B.1.

External validity. Synerise is a single retail platform, so the schema, hard constraints, and validity rules reflect that platform’s tracking conventions (Section 5.3). The finding that $\approx$ one-third of real purchases lack prior same-SKU cart events is a property of this dataset’s logging, not a domain-agnostic claim; generalization to news, media, music, or social feeds would require re-deriving the valid-transition matrix and exposure-prerequisite distribution from domain-appropriate data.

Construct validity. Each fidelity metric was chosen to catch a separable failure mode (Section 7.5), and the results confirm the separation: Iterations 2/3 match popularity inequality (Δ Gini 0.027/0.035 test) yet mildly over-cover (1.19/1.07), while Iteration 1 under-covers (0.65) and concentrates worst (Δ Gini 0.147), a distinction a single aggregate distance would have hidden. The basket-level metrics likewise decompose *where* each architecture’s bias lives: Iteration 1 has the most user-differentiated top- k head yet the worst sampled-distribution concentration, and Iterations 2/3 invert that profile (Section 8.3). The archived TSTR scalar provably hid this distinction, ranking the most concentrated iteration best (Appendix B.1).

Ecological validity. Unchanged by the results: offline evaluation supports claims about static log fidelity, validity rates, and the popularity-bias profile, but not about live deployment with feedback dynamics. No result of Section 8 should be read as predictive of deployment behaviour; whether high-fidelity offline sessions yield useful live traffic remains future work.

9.3 Reliability

Five-seed variance operationalizes test–retest reliability, and the spread is small: $\text{std} \leq 0.003$ for every distributional and rate metric, ≤ 240 SKUs on counts of order 10^5 , and ≤ 0.04 (mainstream) / ≤ 1.3 (niche) on Δ GAP, an order of magnitude below the effects compared. This supports the directional claims of Section 8, except where differences fall below the relevant std, notably the cross-iteration niche- Δ GAP differences (12.6–13.3).

9.4 Limitations

Data-side limitations are in Section 5.3; below, the model and evaluation limitations.

- **Comparison scope.** A single Markov baseline; stronger (GRU/LSTM- or GAN-based) baselines are future work.
- **Single dataset.** Synerise only; no cross-dataset test.
- **Offline-only.** No live deployment or A/B testing.
- **Training cost.** Limits the number of ablations and iterations achievable in the timeline.
- **Throughput.** Autoregressive decoding is sequential per event; whether generation keeps pace with the CMI arrival simulator at deployment scale was not measured (offline-only evaluation), with batched generation the available mitigation.
- **Teacher forcing.** Trained on ground-truth prefixes but generating from its own outputs, so exposure bias can cause drift over long rollouts; scheduled sampling[2] and sequence-level training were not pursued (time).
- **Bias inheritance.** The generator inherits the training data’s popularity skew and temporal selection effects; reporting bias does not remove it.

- **Seasonal non-stationarity.** The test split sits in the early-December regime by construction, so held-out performance conflates generation quality with seasonal extrapolation from the June–November training window.
- **TSTR replaced.** Pre-registered, run, and archived (Appendix B.1): it rewarded mode collapse (Iteration 1’s TSTR-T 0.253 exceeds its own TRTR 0.208) and its vocabulary confound made cross-iteration scores incomparable. The popularity-bias axis (Section 7.4) replaces it.
- **Bias-metric scope.** The user groups are popularity-affinity proxies (no demographics in the data); $\Delta\text{GAP}_{\text{revised}}$ is near-degenerate at e-commerce popularity scales; and the evaluation is a single feedback step rather than the iterated loop of Braun et al. [5], with model iterations standing in for the dynamic axis.

9.5 Ethical Reflection

The ethical requirements of Section 2.5 are revisited here against the pipeline’s measured behaviour, together with the outcomes of a structured ethical peer review: a sit-down session with four fellow Master Applied AI students in which each presented their solution and gave and received ethical feedback on it. The peers raised four points about this pipeline: consent versus anonymisation; an ambiguous societal balance, with the request to name the direct value and its beneficiaries rather than deferring to follow-up research; whether the popularity skew in the data had been taken up as a bias; and the difficulty of embedding human-in-the-loop and inclusivity in a project of this nature. The four paragraphs below answer these points in order. A fifth suggestion, clarifying the ethical stakes through closer stakeholder contact, is answered structurally rather than here: the ethical requirements are stakeholder-owned (Section 4) and the stakeholder loop of Section 3 carried the bias axis into the evaluation. Finally, the lecturer collecting the round cautioned that the brief written feedback should not be read as exhaustive and singled out bias in the synthetic data for deeper study; that depth is what the popularity-bias axis (Section 7.4) subsequently delivered.

Individual. The pipeline uses anonymized aggregate logs and is scoped to aggregate generation; profiling and re-identification are out of scope. Two concerns remain: the source users were likely not specifically informed that their anonymized logs would train a generative model (anonymization is a weaker guarantee than informed consent, acknowledged here but not resolved), and a recommender trained on over-concentrated synthetic data (Section 8.1) can narrow individual exposure and reinforce filter bubbles, which is why the bias group is reported.

Society. The principal risk is feedback-loop amplification: synthetic data inherits popularity skew, and a recommender trained on it can entrench a rich-get-richer dynamic that skews future logs further. The results make this risk measurable, and generator choice has a quantified effect on the skew that would propagate: test-split ΔGini runs from 0.336 (Markov) and 0.147 (Iteration 1) down to 0.027–0.035 for the structurally-primed iterations. The group-level view sharpens this: every iteration reaches niche users with items $\approx 13\times$ their historical popularity level, against +0.45 organically (Section 8.3), so the amplification burden falls on exactly the users with the least mainstream taste. The peers asked for the direct value and its beneficiaries: platforms gain privacy-preserving test data, researchers a reproducible benchmark and reusable evaluation protocol, and end users benefit only indirectly, and only when the bias is controlled.

Bias mitigation. Popularity skew is a measured, first-class property (item coverage, popularity JSD, ΔGini on both splits) and is partially mitigated: the structural priors cut ΔGini from 0.232 to 0.051–0.059 on the training split. It is not eliminated, since popularity JSD stays above the TRTR floor for every iteration.

Oversight and inclusivity. An offline batch pipeline offers no live per-session intervention point, so continuous human-in-the-loop and inclusivity-by-design are hard to embed directly. The substitute is a review gate: the bias report

is a human-readable checkpoint before synthetic data is released, rather than consumed silently. Inclusivity beyond aggregate Gini is addressed at the user-segment level by the group-wise basket metrics (Section 8.3); extending the analysis to demographic groups requires attributes this dataset does not carry and remains future work. This closes the points raised in the peer-review session; the residual tension is retained as the conclusion: synthetic data trained on biased logs inherits those biases, and documentation makes the pipeline more controllable without making it neutral.

10 Conclusion

10.1 Summary of Contributions

This work delivered a schema-constrained, autoregressive session generator for retail e-commerce logs, evaluated under a pre-registered, five-seed protocol against a Markov baseline and a real-vs-real TRTR reference. Three contributions follow. First, a hard-constraint generation-and-repair pipeline producing zero hard-constraint violations at generation time, with a bounded, seed-stable purchase-exposure repair cost (15.6–17.7%). Second, an evaluation that separates fidelity into distinct failure modes and demonstrates empirically that they diverge (coverage versus concentration, sampled-distribution versus top- k -head bias) rather than collapsing into a single distance. Third, a portfolio of three iterations showing that structural priors over a flat softmax reduce popularity over-concentration and calibrate mainstream-user popularity to the real-continuation reference, while niche-user amplification persists at $\approx 13\times$ across all iterations.

Against the requirements of Section 4: the functional and technical requirements are met as validated in Section 4.7; the ethical requirements are operationalized end-to-end, with E1 measured every run, E2’s mitigation levers available in configuration, and the E3 bias report emitted as a standard artefact of every evaluation; the measured niche amplification documents why that gate matters.

10.2 Implications

The central methodological implication is that synthetic-session generators should not be ranked on a single scalar. The archived TSTR evaluation (Appendix B.1) is a worked example: Iteration 1 scored best on the headline utility metric precisely because it had failed upstream on popularity concentration. Practitioners should report group-level popularity-bias metrics against a real-behaviour reference and treat a synthetic-exceeds-real utility score as a warning sign rather than a success. For the present pipeline, the structurally-primed iterations are the defensible choice, popularity-calibrated for mainstream users, with the persistent $\approx 13\times$ niche amplification the open problem.

10.3 Recommendations and Future Work

Four extensions follow directly from the limitations and the construct-validity discussion. (i) Iterated feedback-loop bias measurement in the dynamic setting of Braun et al. [5], with demographic or segment-level group data where available, and targeted niche-user debiasing (e.g. popularity-calibrated sampling) as the concrete next mitigation. (ii) A stronger generator baseline (GRU/LSTM- or GAN-based) that can condition on user histories, so the bias axis gains a contesting comparator. (iii) Cross-dataset and cross-domain replication, re-deriving the valid-transition matrix and exposure-prerequisite distribution per domain. (iv) Live or simulated-feedback evaluation to test whether offline fidelity predicts useful synthetic traffic under deployment dynamics. A narrower open question is whether the mild over-coverage of the structurally-primed iterations (1.19/1.07) is structural to factored item heads or tunable through the SVD-PQ sampling temperature, token count, and codebook size.

References

- [1] ABDOLLAHPOURI, H., MANSOURY, M., BURKE, R., AND MOBASHER, B. The unfairness of popularity bias in recommendation. In *Workshop on Recommendation in Multi-stakeholder Environments (RMSE) at RecSys '19* (2019). arXiv:1907.13286.
- [2] BENGIO, S., VINYALS, O., JAITLY, N., AND SHAZEER, N. Scheduled sampling for sequence prediction with recurrent neural networks, 2015.
- [3] BOBADILLA, J., GUTIÉRREZ, A., YERA, R., AND MARTÍNEZ, L. Creating synthetic datasets for collaborative filtering recommender systems using generative adversarial networks. *Knowledge-Based Systems* (2023).
- [4] BOUNTOURIDIS, D., HARAMBAM, J., MAKHORYKH, M., MARRERO, M., TINTAREV, N., AND HAUFF, C. Siren: A simulation framework for understanding the effects of recommender systems in online news environments. In *Proceedings of the ACM Conference on Fairness, Accountability, and Transparency (FAT*)* (2019).
- [5] BRAUN, V., BHAUMIK, D., AND DEY, D. Metrics for popularity bias in dynamic recommender systems. In *Proceedings of the 15th International Joint Conference on Knowledge Discovery, Knowledge Engineering and Knowledge Management (KDIR)* (2023). arXiv:2310.08455.
- [6] BROWN, S. The c4 model for software architecture. <https://static.simonbrown.jp/pt2024-c4.pdf>, 2024. Presentation slides, accessed 2026-05-11.
- [7] CHANEY, A. J. B. Recommendation system simulations: A discussion of two key challenges. *arXiv preprint arXiv:2109.02475* (2021).
- [8] CRESWELL, A., WHITE, T., DUMOULIN, V., ARULKUMARAN, K., SENGUPTA, B., AND BHARATH, A. A. Generative adversarial networks: An overview. *IEEE Signal Processing Magazine* 35, 1 (2018), 53–65.
- [9] CUI, Y., JIA, M., LIN, T.-Y., SONG, Y., AND BELONGIE, S. Class-balanced loss based on effective number of samples, 2019.
- [10] EKSTRAND, M. D., CHANEY, A., CASTELLS, P., BURKE, R., ROHDE, D., AND SLOKOM, M. Simurec: Workshop on synthetic data and simulation methods for recommender systems research. In *Proceedings of the 15th ACM Conference on Recommender Systems (RecSys '21)* (2021), pp. 803–805.
- [11] ESTEBAN, C., HYLAND, S. L., AND RÄTSCHE, G. Real-valued (medical) time series generation with recurrent conditional gans, 2017.
- [12] EUROPEAN PARLIAMENT AND COUNCIL OF THE EUROPEAN UNION. Regulation (eu) 2016/679 of the european parliament and of the council of 27 april 2016 on the protection of natural persons with regard to the processing of personal data (general data protection regulation). *Official Journal of the European Union L 119* (2016), 1–88.
- [13] FIGUEIRA, A., AND VAZ, B. Survey on synthetic data generation, evaluation methods and gans. *Mathematics* 10, 15 (2022), 2733.
- [14] GU, J., BRADBURY, J., XIONG, C., LI, V. O. K., AND SOCHER, R. Non-autoregressive neural machine translation, 2018.
- [15] HALFAKER, A., KEYES, O., KLUVER, D., THEBAULT-SPIEKER, J., NGUYEN, T., SHORES, K., UDUWAGE, A., AND WARNCKE-WANG, M. User session identification based on strong regularities in inter-activity time, 2019.
- [16] HIDASI, B., KARATZOGLOU, A., BALTRUNAS, L., AND TIKK, D. Session-based recommendations with recurrent neural networks. *arXiv preprint arXiv:1511.06939* (2015).
- [17] HURLEY, N. P., AND RICKARD, S. T. Comparing measures of sparsity, 2009.
- [18] JAKOMIN, M., PEVEC, D., KONONENKO, I., AND BOSNIĆ, Z. Generating interdependent data streams for recommender systems. *Expert Systems with Applications* 112 (2018), 473–484.
- [19] JANG, E., GU, S., AND POOLE, B. Categorical reparameterization with gumbel-softmax, 2017.
- [20] JÉGO, H., DOUZE, M., AND SCHMID, C. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128. INRIA Research Report inria-00514462v2.
- [21] JORDON, J., SZPRUCH, L., HOUSSEAU, F., BOTTARELLI, M., CHERUBIN, G., MAPLE, C., COHEN, S. D., AND WELLER, A. Synthetic data: What, why and how? *arXiv preprint arXiv:2205.03257* (2022).
- [22] JR., F. J. M. The kolmogorov-smirnov test for goodness of fit. *Journal of the American Statistical Association* 46, 253 (1951), 68–78.
- [23] KANG, W. C., AND MCAULEY, J. Self-attentive sequential recommendation. In *Proceedings of the IEEE International Conference on Data Mining (ICDM)* (2018).
- [24] LIN, J. Divergence measures based on the shannon entropy. *IEEE Transactions on Information Theory* 37, 1 (1991), 145–151.
- [25] LU, Y., SHEN, A., WANG, H., WANG, X., VAN RECHEM, C., AND FU, W. Machine learning for synthetic data generation: A review. *arXiv preprint* (2024).
- [26] LUCHERINI, E., SUN, M., WINECOFF, M., AND NARAYANAN, A. T-recs: A simulation tool to study the societal impact of recommender systems. *arXiv preprint arXiv:2107.08959* (2021).
- [27] MCINERNEY, J., BROST, B., CHANDAR, P., MEHROTRA, R., AND CARTERETTE, B. Accordion: A trainable simulator for long-term interactive systems. In *Proceedings of the 15th ACM Conference on Recommender Systems (RecSys)* (2021).
- [28] MENG, Z., MCCREADIE, R., MACDONALD, C., AND OUNIS, I. Exploring data splitting strategies for the evaluation of recommendation models, 2020.
- [29] MENON, A. K., JAYASUMANA, S., RAWAT, A. S., JAIN, H., VEIT, A., AND KUMAR, S. Long-tail learning via logit adjustment. *arXiv preprint arXiv:2007.07314* (2021).
- [30] MIN, B. Project wiki: Onderzoek, ontwerp, ontwikkeling, evaluatie, feedback & iteratie. <https://gitlab.fdmci.hva.nl/maai/2025-2026-sem2/centre-for-market-insights/beau-min-maai-portfolio/-/wikis/pages>, 2026. GitLab wiki, accessed 2026-06-12.
- [31] MIN, B. Session-generator pipeline: source code and documentation. <https://gitlab.fdmci.hva.nl/maai/2025-2026-sem2/centre-for-market-insights/beau-min-maai-portfolio>, 2026. GitLab repository, code/pipeline, accessed 2026-06-12.
- [32] MORIN, F. Hierarchical probabilistic neural network language model. *AISTATS 2005 - Proceedings of the 10th International Workshop on Artificial Intelligence and Statistics* (01 2005).
- [33] MÜLLER, R., KORNBLITH, S., AND HINTON, G. When does label smoothing help? In *Advances in Neural Information Processing Systems (NeurIPS)*

- (2019).
- [34] PETROV, A. V., AND MACDONALD, C. Generative sequential recommendation with gptrec. *arXiv preprint arXiv:2306.11114* (2023).
 - [35] RODRIGUEZ-HERNANDEZ, M. C., ILARRI, S., TRILLO-LADO, R., AND HERMOSO, R. Datagencars: A generator of synthetic data for the evaluation of context-aware recommendation systems. *Pervasive and Mobile Computing* 38 (2017), 516–541.
 - [36] STAVINOVA, E., GRIGORIEVSKIY, A., VOLODKOVICH, A., CHUNAEV, P., BOCHENINA, K., AND BUGAYCHENKO, D. Synthetic data-based simulators for recommender systems: A survey. *arXiv preprint arXiv:2206.11338* (2022).
 - [37] SUN, F., LIU, J., WU, J., PEI, C., LIN, X., OU, W., AND JIANG, H. Bert4rec: Sequential recommendation with bidirectional encoder representations from transformer. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM)* (2019).
 - [38] SYNERISE. Synerise recsys challenge 2025. <https://github.com/Synerise/recsys2025>, 2025. GitHub repository, accessed 2026-05-11.
 - [39] SZEGEDY, C., VANHOUCKE, V., IOFFE, S., SHLENS, J., AND WOJNA, Z. Rethinking the inception architecture for computer vision. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2016).
 - [40] VASWANI, A., SHAZEER, N., PARMAR, N., USZKOREIT, J., JONES, L., GOMEZ, A. N., KAISER, L., AND POLOSUKHIN, I. Attention is all you need, 2023.
 - [41] WILLIAMS, R. J., AND ZIPSER, D. A learning algorithm for continually running fully recurrent neural networks. *Neural Computation* 1, 2 (1989), 270–280.
 - [42] ZHAO, X., XIA, L., ZOU, L., LIU, H., YIN, D., AND TANG, J. Usersim: User simulation via supervised generative adversarial network. In *Proceedings of The Web Conference (WWW)* (2021).

A Model Architecture Diagrams

This appendix collects the architecture diagrams that are referenced from Sections 6.2 and 6.3. Figure 15 gives the component-level view of the generation runtime that hosts all three iterations. The remaining figures show the item-prediction branch for each iteration; all three share the same transformer backbone, action head, temporal head, and cross-session memory and differ only in how the next SKU is predicted from the shared hidden state.

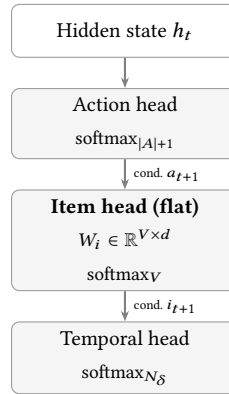


Fig. 16. Iteration 1. A single V -way softmax predicts the SKU directly from h_t , conditioned only on the sampled action.

Component View: Session Generator Pipeline - Generation runtime
 Component view of the generation runtime: arrival input drives the embedding, the transformer backbone produces a shared hidden state, three factored heads sample an action / item / dt, and the validity layer gates the output log.

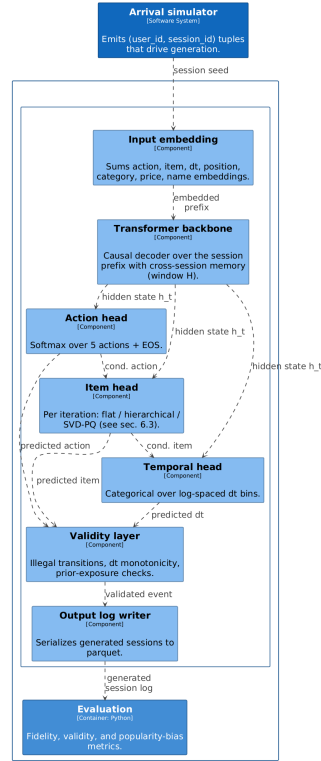


Fig. 15. Component view of the generation runtime (C4 component diagram). The arrival simulator drives the input embedding; the transformer backbone produces a shared hidden state; three factored heads sample an action, an item, and an inter-event time; the validity layer gates the output log. Per-iteration item-head variants are described in Section 6.3; the validity layer is specified in Section 6.4.

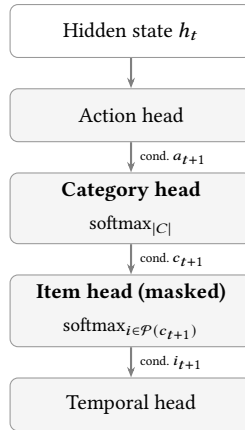


Fig. 17. Iteration 2. A category head intervenes between h_t and the item softmax; the item softmax is masked to the SKU pool of the sampled category.

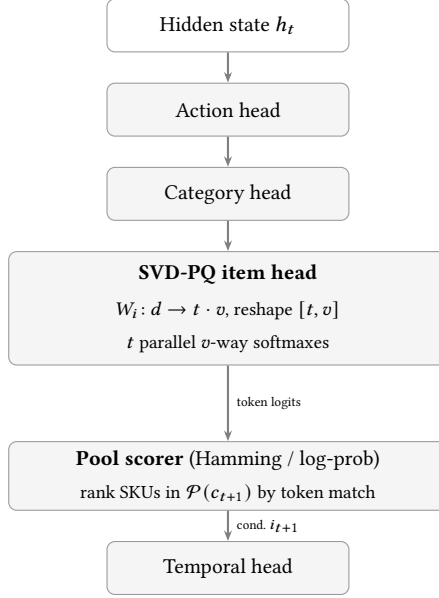


Fig. 18. Iteration 3. The item head emits t parallel v -way token softmaxes; an offline SVD-PQ codebook maps each SKU to a token tuple, and an in-pool scorer maps the token logits back to a single SKU at inference time.

B Additional Experimental Details

The fidelity metric suite referenced from Section 7.2 is reproduced in full in Table 14.

B.1 Archived Downstream-Utility (TSTR) Evaluation

The pre-registered utility axis followed the Train-Synthetic-Test-Real protocol [11, 21]: a fixed downstream evaluator (GRU4Rec [16], identical architecture, hyperparameters, and early stopping across all conditions) was trained on the transformer’s synthetic sessions (TSTR-T), the Markov baseline’s (TSTR-M), and the real training split (TRTR, an interpretive ceiling), and evaluated on the real held-out test split with HR@10 and Normalized Discounted Cumulative Gain@10 (NDCG@10). GRU4Rec was chosen over transformer evaluators such as SASRec [23] or BERT4Rec [37] because it is the canonical session-based baseline [16], and because an evaluator that does not share the generator’s transformer inductive biases keeps the judge architecturally independent of the generators it ranks. A shared-vocabulary filter kept the three conditions comparable per iteration; its intersection with each generator’s emitted SKU set produces the differing OOV rates in Table 15.

The transformer dominates Markov throughout ($\approx 14\text{--}30\times$ HR@10), but the metric failed as a cross-iteration ranking. Iteration 1’s TSTR-T (0.253) exceeds its own TRTR (0.208): a generator should not train a recommender better than the real data it imitates, and the excess is explained by its concentrated popularity yielding an artificially easy next-item task. Iterations 2 and 3 fall below TRTR (the honest direction) but are mutually indistinguishable (0.124 vs. 0.124) at differing OOV rates (0.393/0.413), so absolute TSTR is not comparable across iterations.

An exploratory per-iteration correlation between each fidelity metric and TSTR-T HR@10 across the five seeds yields no relationship that is both significant and stable across iterations ($n = 5$ per coefficient). Two coefficients

Table 14. Fidelity metric suite. Distributional metrics compare generated sessions to the training distribution; behavioral-rate metrics compare session-level event composition; bias metrics are informational and diagnose item-side collapse against a size-matched real subsample. Lower is better for all metrics except where indicated. All metrics are implemented in `evaluation/fidelity.py`.

Group	Metric	Range	Description
Distributional	JSD(action distribution)	[0, 1]	Jensen–Shannon divergence between real and generated action-type frequencies (5 event types). Marginal-level behavioral mix [24].
	KS(session length)	[0, 1]	Two-sample Kolmogorov–Smirnov statistic on session length distributions. Engagement-shape match.
	L1(action bigram matrix)	[0, 2]	L1 distance between normalized real and generated action bigram transition matrices. Sequential structure of actions, not just marginals.
	KS(temporal δ_t)	[0, 1]	Two-sample KS on inter-event delta-time distributions in seconds. Pacing realism.
	Sample diversity (Jaccard ratio)	ratio, 1.0 ideal	Mean pairwise Jaccard distance over item sets in synthetic sessions, normalized by the same statistic on real sessions. Reported as a ratio so the sparse-catalog Jaccard floor cancels; < 1 means synthetic sessions are too repetitive across users, > 1 means too diffuse.
Behavioral rate	Δ conversion rate	[0, 1]	Absolute difference between the fraction of synthetic and real sessions containing at least one <code>product_buy</code> . Funnel endpoint match.
	Δ cart abandonment rate	[0, 1]	Absolute difference between the fraction of sessions containing <code>add_to_cart</code> but no <code>product_buy</code> . Mid-funnel drop-off.
Bias (informational)	Item coverage	ratio, 1.0 ideal	Unique synthetic SKUs divided by unique SKUs in the size-matched real subsample of the training reference. Diagnoses head-only generation.
	Popularity JSD	[0, 1]	JSD between synthetic and matched-real per-SKU popularity distributions over the union of SKU keys (no top- K truncation).
	Δ Gini	[0, 1]	Absolute difference between the Gini coefficients of the synthetic and matched-real SKU frequency distributions. Concentration parity.

Table 15. Archived downstream utility, five-seed means $\pm$ std. HR@10 and NDCG@10 higher is better. The OOV rate differs across iterations because the shared-vocabulary filter intersects with the synthetic SKU set produced by each generator.

Iteration	Condition	HR@10	NDCG@10	OOV
Iter 1	TSTR-T	0.253 ± 0.004	0.227 ± 0.004	0.451
	TSTR-M	0.008 ± 0.002	0.006 ± 0.001	0.451
	TRTR	0.208 ± 0.006	0.180 ± 0.005	0.451
Iter 2	TSTR-T	0.124 ± 0.006	0.103 ± 0.005	0.393
	TSTR-M	0.009 ± 0.002	0.006 ± 0.001	0.393
	TRTR	0.189 ± 0.006	0.163 ± 0.005	0.393
Iter 3	TSTR-T	0.124 ± 0.005	0.102 ± 0.005	0.413
	TSTR-M	0.009 ± 0.002	0.006 ± 0.001	0.413
	TRTR	0.195 ± 0.006	0.169 ± 0.005	0.413

reach $p < 0.05$, both in Iteration 1 (KS session length $r = -0.89$, item coverage $r = -0.89$), but neither replicates in Iterations 2–3, and the $\text{KS}(\delta_t)$ coefficient is undefined because that metric has zero seed-to-seed variance.

Because the scalar rewarded exactly the popularity concentration the bias metrics expose, ranking the most concentrated iteration (Iteration 1) best, the utility axis was replaced by the popularity-bias evaluation of Section 7.4; these results are retained for the pre-registered pass conditions (Section 8.4).

C Hyperparameters

Table 16. Tunable hyperparameters of the pipeline, grouped by stage. Configuration keys correspond verbatim to entries in config.yaml.

Group	Config key	Description
Sessionization	session_timeout_min	Inactivity gap that starts a new session (min).
	history_window	Cross-session memory window H (events).
	train_cutoff	Train/validation split date (session-start).
	val_cutoff	Validation/test split date (session-start).
Vocabulary	vocab_k	Top- K item vocabulary cap.
Temporal head	n_temporal_bins	Number of log-spaced δ_t bins.
	temporal_min_s	Minimum representable inter-event delta (s).
Transformer	train_d_model	Hidden dimension d .
	train_n_layers	Number of decoder layers L .
	train_n_heads	Attention heads (must divide d).
	train_max_length	Max events per session, including EOS.
	train_batch_size	Mini-batch size.
	train_lr	AdamW learning rate.
	train_epochs	Maximum training epochs.
	train_patience	Epochs without val-loss improvement before stop.
Category head	category_rare_threshold	SKU count below which a category is collapsed to RARE.
SVD-PQ item head	svdpq_t	Tokens per SKU.
	svdpq_v	Bins per token dimension.
	svdpq_label_smoothing	Per-dim CE label smoothing.
	svdpq_binning	quantile or uniform.
	svdpq_noise_std	Pre-bin Gaussian noise stddev.
	svdpq_min_interactions	Cold-SKU fallback threshold.
Loss weighting	loss_weights_enabled	Enable per-head loss weighting.
	loss_weights	4-entry vector: action, category, item, temporal.
Bias mitigation	item_loss_freq_weight_alpha	Inverse-frequency item-CE exponent α .
	cat_logit_adj_tau	Menon logit-adjustment strength τ .
Inference	infer_temperature	Action / category / temporal softmax temperature.
	infer_item_temperature	Item-head sampling temperature.
	infer_pool_temperature	SVD-PQ in-pool softmax temperature.
	svdpq_infer_scorer	hamming or log_prob pool scorer.
Final evaluation	final_eval_n_sessions	Synthetic sessions per evaluation seed.
	final_eval_seeds	Random seeds for multi-seed reporting.
GRU4Rec downstream	gru4rec_embed_dim	Item embedding size.
	gru4rec_hidden_dim	GRU hidden size.
	gru4rec_max_epochs	Maximum training epochs.
	gru4rec_batch_size	Mini-batch size.
	gru4rec_lr	Adam learning rate.
	gru4rec_patience	Early-stopping patience.